

Заняття 1. Розгортання frontend-частини проєкту.

Актуальність та цілі заняття

Заняття 1. «Розгортання frontend-частини

проєкту. Тренування з викладачем»

ХІД ЗАНЯТТЯ

Вітаємо на першому практичному занятті, яке відкриває наш курс «Практикум з сучасних методологій розробки ПЗ». Сьогодні ми переходимо від теорії до дії, і це дійсно захоплююче!

Твоє перше завдання — стати архітектором та будівельником: навчитися «читати» технічне завдання та дизайн-макет, як справжній професіонал, і закласти міцний фундамент для фулстек-проєкту. Ти розгорнеш frontend-частину і створиш якісну, семантичну розмітку сторінки. Пам'ятай, якісна верстка — це запорука успіху всього проєкту!

Тож, приготуй IDE, налаштуйся на роботу з Figma, і поїхали створювати щось класне!

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

У роботі над проєктами важливо вміти виконувати грамотно технічне завдання, розуміти принципи верстки сторінок згідно з макетом у Figma.

ЦІЛІ ЗАНЯТТЯ

Навчитися розгортати новий проєкт і готувати матеріали для нього, а також попрактикуватись створювати семантичну розмітку проєкту згідно з макетом.

ОСНОВНІ ЗАВДАННЯ

1. Навчитись аналізувати дизайн-макет і планувати розмітку сайту.
2. Розібрати як правильно розбивати проєкт на правильну структуру та секції.
3. Реалізувати верстку згідно з [макетом](#).
4. Перевірити реалізований код у валідаторі та виправити допущені помилки.

Підготовка до заняття

Перейди за посиланням на [макет](#), з яким будемо працювати та проаналізуй детальніше його і [вимоги](#) до його розробки, а також критерії прийому проєкту.

Пам'ятай!

Розмітка майбутньої вебсторінки повинна мати смислове значення, як сторінка газети або журналу. На сторінці є секції, заголовки, списки, зображення, абзаци тексту тощо. Для опису всього цього багатства типів контенту існують відповідні теги. Тобто семантика спочатку є частиною HTML, але вона марна, якщо реалізована неправильно.

Нагадаємо, що **семантична розмітка** — це підхід до створення HTML-розмітки, що ґрунтується на використанні тегів за їх призначенням відповідно до специфікації й, які відповідають типу контенту, що розмічається.

Валідна верстка — це верстка, яка зроблена за всіма стандартами W3C (www consortium). Тобто верстку можна вважати валідною лише тоді, коли вона буде відповідати встановленим W3C стандартам. Валідація потрібна для виявлення помилок у синтаксисі розмітки HTML-документа і розбіжностей з HTML-специфікацією, вказаною в <!DOCTYPE >.

У коді важлива кожна кома, лапки та дужка, ім'я атрибута і тегу, відкриваючий і закриваючий тег. Очима, особливо новачкам, за усім цим не встежити, тому придумали **валідатор** – програму для такої перевірки. В результаті валідації документ або проходить валідацію або отримує список рекомендованих виправлень.

Увага!

Валідатори перевіряються тільки HTML-синтаксис і відповідають специфікації, а не семантику. Семантику можна перевірити в правилах <https://html.spec.whatwg.org/>

Для перевірки зробленої розмітки сторінки [макету](#) використовують [спеціальний онлайн-сервіс \(валідатор\)](#).



Для детальнішого повторення матеріалів попередніх дисциплін, необхідних для виконання даного етапу проєкту, рекомендуємо ознайомитись із темами 1-4 дисципліни «Основи веброзробки (мови розмітки)»:

[Тема 1. Основи web-розробки](#)

[Тема 2. Інструменти розробки](#)

[Тема 3. Знайомство з HTML](#)

[Тема 4. Структура HTML-документа](#)

та практичним заняттям «[Основи веброзробки \(Java Script\)](#)»

Заняття 2. Розгортання frontend-частини проєкту.

Актуальність та цілі заняття

Заняття 2. «Розгортання frontend-частини проєкту. Практика»

ХІД ЗАНЯТТЯ

Ми успішно завершили перший етап — створили початкову розмітку нашого проєкту. Сьогодні ми переходимо до найважливішої частини навчання: **аналізу та вдосконалення!**

Це заняття присвячене виключно твоїм питанням, твоєму коду і твоїм перемогам (навіть над найменшими помилками). Ми проведемо справжнє «бойове хрещення» коду: розберемо типові помилки, подивимося, як працюють професійні інструменти для валідації та, найголовніше, навчимося робити **Code Review** на практиці.

Не бійся показувати свій код, адже кожна знайдена помилка — це крок до рівня Middle-розробника. Тож, починаймо перевірку: валідатори, DevTools та Kahoot! вже чекають на тебе!

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Для закріплення навичок аналізу ТЗ та вимог важливо зробити розмітку сторінки, правильно спланувати її структуру та перевірити реалізовану розмітку у валідаторі, щоб простежити власні помилки.

ЦІЛІ ЗАНЯТТЯ

Вміти розгортати новий проєкт і готувати матеріали для нього, створювати семантичну розмітку проєкту згідно макету.

ОСНОВНІ ЗАВДАННЯ

1. Потренуватись аналізувати дизайн-макет і планувати розмітку сайту.
2. Створити структуру сайту та секції у проєкті.
3. Реалізувати розмітку сайту згідно альтернативного [макету](#) Flora.
4. Перевірити реалізований код у валідаторі та виправити допущені помилки.

Підготовка до заняття

1. Аналіз макету, вимоги та розгортання проєкту

На даному етапі виконання свого проєкту необхідно виконати такі завдання:

- 1) Завершити HTML-розмітку макета сторінки з [макету](#), без CSS оформлення, якщо ви ще її не завершили.
- 2) Запустити код у створений репозиторій UMT-markup-practice-ПІБ.

3) Перевірити [GitHub Pages](#) і додай посилання на живу сторінку в секцію [About](#) GitHub-репозиторія.

Проект на цьому етапі повинен відповідати **таким вимогам:**

1) HTML-файл називається `index.html`.

2) У корені проєкту є папка `images` з зображеннями.

3) У назвах файлів відсутні великі літери, кирилиця, пробіли й трансліт. Назви містять тільки літери й слова англійської мови.

4) Вихідний код відформатований за допомогою Prettier.

5) Всі зображення та текстовий контент взяті з макета.

6) Всі растрові зображення оптимізовані з використанням [squoosh](#).

7) Код написаний з дотриманням [настанови](#).

8) Розмітка набрана у файлі `index.html`.

9) Виконана HTML-розмітка всіх елементів макета.

10) Теги використані відповідно до їх семантичного змісту.

11) HTML проходить перевірку [валідатором](#) без помилок.

12) У розмітці є теги для виділення основної структури сторінки: `<header>`, `<main>` і `<footer>`.

13) Логотип в хедері й футері — це посилання.

- 14) Тег <nav> використаний один раз на сторінці — в хедері.
- 15) Тег <h1> використано один раз на сторінці.
- 16) Елементом-кнопкам задано правильний type.
- 17) Заголовки секцій розмічені тегом <h2>.
- 18) У тегів вказані атрибути розмірів, як мінімум width.
- 19) У тегів є атрибут alt, який заповнений коротким описом про те, що зображено на малюнку. Опис тією ж мовою що і решта текстового контенту.
- 20) Зображення експортовані з макета у форматі jpg.
- 21) Групи однотипних елементів зібрані у списки .

2. Оформлення коду

В процесі роботи над створенням вебсайтів завжди необхідно повертатися до вже написаного коду для внесення змін. В переважній більшості проєктів працюють більше одного розробника, відповідно якщо код не оформлений відповідно до стандартів – він втрачає читабельність і, як наслідок – швидкість розробки падає.

КОРИСНО

Використовуй це [керівництво від @mdo](#) для написання коду в процесі вивчення технологій.

Для зручності розробки сайту використовують коментарі для позначення секцій у розмітці та стилях, щоб краще орієнтуватися в коді.

Коментарі в HTML-коді задаються так:

```
<!-- будь-який текст чи код -->
```

Текст всередині коментаря не відображається браузером на сторінці.

Коментарі зазвичай використовуються в наступних випадках:

Для коментування коду. Завжди корисно залишити підказку.
Для тимчасового відключення коду. Видаляти код незручно, оскільки його треба буде відновлювати, а закоментувати й потім розкоментувати — найкраще рішення.

Коментарі можна використовувати в будь-якому місці сторінки, крім тегу — всередині нього вони не працюють.

Проте! Коли проєкт повністю готовий, коментарі з коду варто почистити.

Також варто форматувати код, використовуючи плагін Prettier Code Formatter.

3. Інструменти розробника

Нагадаємо, що для ефективної розробки потрібні інструменти, зокрема і для написання HTML-коду. Тому необхідно пригадати

як працювати з інструментами розробника. Вони присутні у всіх сучасних браузерах і не вимагають додаткового встановлення.

Інструменти розробника — це вбудований функціонал браузера, який дозволяє отримувати інформацію про вихідний код сторінки, стилі, мережеві запити й багато іншого.

Інструменти розробника можна **викликати наступними методами:**

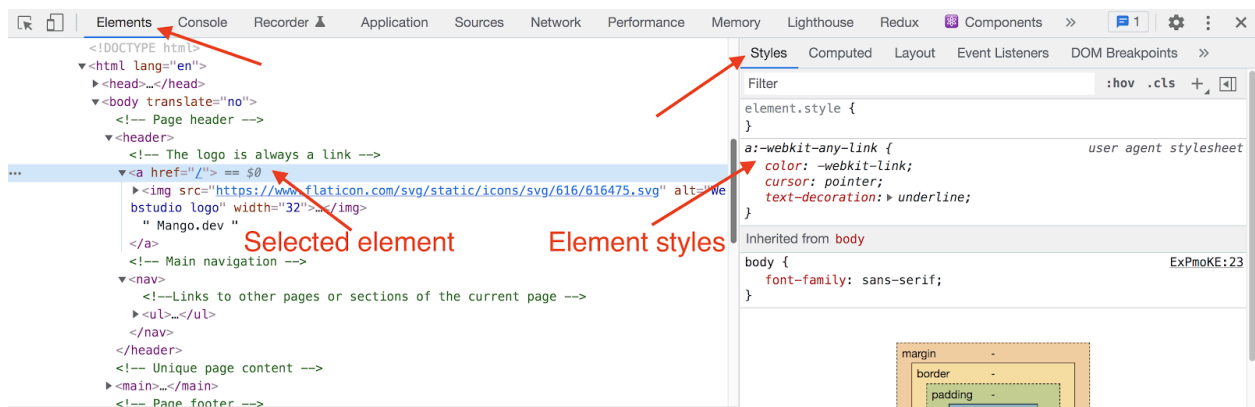
Натиснувши клавішу F12

Комбінацією клавіш Ctrl+Shift+I для Windows

Комбінацією клавіш Cmd+Opt+I для MacOS

На вебсторінці натиснути правою кнопкою миші й вибрати пункт меню «Переглянути код»

До того ж, наводячи курсор миші на певний елемент, він буде підсвічуватися у вікні браузера (viewport).



Крім інспектування елементів, інструменти розробника дозволяють здійснити зміни в HTML і CSS-коді. Такі зміни відразу будуть відображені на вебсторінці, але у вихідному коді

зберігатися не будуть. Ця функція зручна для експериментів з розміткою і стилями.

Радимо ознайомитись: [Огляд всіх можливостей DevTools](#).

Підсумки

Сьогодні ви потренувались розгорнути проєкт та створити розмітку сторінки, а саме:

- 1) навчились аналізувати макет та планувати верстку сторінок сайту;
- 2) розібрали як правильно розбивати проєкт на правильну структуру/секції;
- 3) реалізували верстку згідно з макетом;
- 4) перевірили реалізований код у валідаторі, виправили допущені помилки.

На наступному занятті зробимо нормалізацію і скидання стилів та будемо працювати над стилізацією шрифтів.

Заняття 3. Стилiзацiя шрифтiв та скидання стилiв.

Актуальність та цілі заняття

Заняття 3. «Стилізація шрифтів, нормалізація та скидання стилів. Тренування з викладачем»

ХІД ЗАНЯТТЯ

Ми успішно завершили найважливіший етап — створення міцного семантичного каркаса проєкту. Тепер час вдихнути в нього життя і зробити його візуально привабливим!

Сьогодні ми сфокусуємося на двох ключових елементах професійної верстки: **стилізації шрифтів** та **нормалізації стилів**.

Навчимося правильно підключати шрифти згідно з макетом, зрозуміємо, чому **line-height** має бути коефіцієнтом, і як уникнути «сюрпризів» від браузерів за допомогою **normalize.css**.

Якість вашого CSS визначає не лише красу, а й легкість підтримки коду. Тож, готуйтеся працювати з **body**, каскадами та блоковою моделлю, щоб створити ідеальний візуальний фундамент.

Починаймо!

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Розуміння базових принципів та етапів CSS-стилізації сайту згідно з макетом полегшує процес розробки проєкту, тому важливо розібратися як правильно працювати зі стилями згідно з макетом.

ЦІЛІ ЗАНЯТТЯ

Пригадати як використовувати CSS-селектори, нормалізацію та властивості для стилізації шрифтів.

ОСНОВНІ ЗАВДАННЯ

1. Пригадати логіку роботи стилів у браузерях і навчитися впливати на їхню поведінку, працюючи з блоковою моделлю документа, геометрією елементів та типами тегів.
2. Пригадати як підключати шрифти та як брати їх з дизайн-макета для стилізації.
3. Реалізувати нормалізацію стилів та стилізацію шрифтів згідно [з макетом](#).

Підготовка до заняття

1. Аналіз шрифтів на макеті та їх підключення

Пригадаємо, що шрифти поділяють на **платні** й **безкоштовні**.

Платні шрифти можна використовувати лише після купівлі ліцензії — інакше це порушує авторські права.

Проте є багато гарних безкоштовних шрифтів, які знаходяться в загальному доступі.

[Google Fonts](#) — це сховище величезної кількості безкоштовних шрифтів. Все що необхідно зробити – це знайти потрібний

шрифт, отримати на нього посилання й увімкнути у своєму HTML-файлі як ще одну таблицю стилів.

Таким чином, шрифти можна **увімкнути мінімум двома способами:**

1. Через CDN з видаленого сервера (наприклад Google Fonts):

HTML:	<code><link href="https://fonts.googleapis.com/css2?family=Montserrat:wght@400;700&display=swap" rel="stylesheet"></code>
CSS:	<code>font-family: 'Montserrat', serif;</code>

2. Локально, попередньо завантаживши потрібний шрифт на комп'ютер:

Синтаксис:	<code>@font-face { font-family: 'Wisdom Script'; src: url(../fonts/Wisdom_Script_AJ.otf); src: url(../fonts/WisdomScriptAJ.ttf) format("ttf"); }</code>
-------------------	---

Аналізуємо текст та типи шрифтів на [макеті](#), підключаємо обрані шрифти через Google-fonts.

Заняття 4. Стилiзацiя шрифтiв та скидання стилiв.

Актуальність та цілі заняття

Заняття 4. «Стилізація шрифтів, нормалізація

та скидання стилів. Практика»

ХІД ЗАНЯТТЯ

Доброго дня! Ми пройшли теорію: навчилися правильно підключати шрифти, працювати з коефіцієнтом `line-height` та запобігати сюрпризам від браузерів за допомогою нормалізації.

Сьогодні час застосувати ці знання на практиці й побачити їхній реальний ефект у проєкті Flora. Це буде заняття максимальної концентрації, присвячене **розбору твого коду та виявленню тих дрібниць**, які відрізняють аматорську верстку від професійної.

Ми перевіримо: чи коректно були під'єднані шрифти, чи працює нормалізація і чи всі кольори винесені у CSS-змінні. Готуй свій проєкт — сьогодні ми закріплюємо наші навички CSS-гігієни!

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Розуміння потоку документа, будови HTML-елементів, а також принципів підключення та стилізації шрифтів дає змогу писати якісніший і структурований код. Саме тому важливо закріпити на практиці матеріал, який ми пригадали на минулому уроці.

ЦІЛІ ЗАНЯТТЯ

Потренуватися визначати, підключати та стилізувати шрифти згідно [з макетом](#) у розгорнутий проєкт і навчитися додавати підключення нормалізації та скидання стилів.

ОСНОВНІ ЗАВДАННЯ

1. Закріпити навички аналізу шрифтів на дизайн-макеті.
2. У проєкті з [макету](#) підключити та стилізувати шрифти.
3. Додати підключення нормалізації та скидання стилів.

Підготовка до заняття

1. Підключення та стилізація шрифтів в розгорнутому проєкті

Для закріплення знань на практиці виконай такі завдання:

- 1) В репозиторії `UMT-markup-practice-ПІБ` виконай оформлення шрифтів з макету сторінки з [макету](#).
- 2) Для оптимізації зображень використовуй сервіс [squoosh](#).
- 3) Налаштуй `GitHub Pages` і додай посилання на живу сторінку в секцію `About` GitHub-репозиторію.

Виконаний проєкт на даному етапі має відповідати таким

вимогам:

- 1) У корені проєкту є папка `images` з зображеннями.

- 2) У корені проєкту є папка **CSS** з файлом стилів.
- 3) Всі стилі написані в одному файлі **styles.css**, який знаходиться в папці **CSS**.
- 4) У назвах файлів відсутні великі літери, пробіли й трансліт. Назви містять тільки літери й слова англійської мови.
- 5) Вихідний код відформатований за допомогою Prettier.
- 6) Всі зображення та текстовий контент взяті з макета.
- 7) Всі растрові зображення оптимізовані з використанням [squoosh](#).
- 8) Код написаний з дотриманням [настанови](#).
- 9) Виконана HTML-розмітка всіх елементів макета.
- 10) Теги використані відповідно до їх семантичного змісту.
- 11) HTML проходить перевірку [валідатором](#) без помилок.
- 12) Імена класів описові та зрозумілі іншому розробнику.
- 13) Імена класів не містять великих літер, пробілів, трансліту і назв тегів. Імена містять тільки літери та слова англійської мови. Якщо ім'я класу складається з декількох слів, вони розділяються дефісом.
- 14) У тегів **** вказані атрибути розмірів, як мінімум **width**.
- 15) Зображення експортовані з макета у форматі **jpg**.

16) Групи однотипних елементів зібрані у списки `<ul>`.

17) Всі необхідні шрифти за макетом та їх варіації (вага і накреслення) підключені з сервісу Google Fonts одним посиланням.

18) Для оформлення використовуються селектори класу.

19) В стилях відсутній `!important`.

20) Інтерактивні елементи (кнопки й посилання), при наведенні мишкою або фокусі з клавіатури, мають активний стан, вказаний в [Style guide](#) (зміна кольору).

21) Текст контактів в хедері змінює колір при ховері та фокусі.

22) Для елемента `<body>` задана властивість `font-family` з домінантним шрифтом на макеті.

23) Вказані альтернативні варіанти шрифту й тип сімейства (без зарубок) в кінці переліку `font-family` в елемента `<body>`.

24) Домінуюче сімейство шрифтів явно задане тільки для елемента `<body>`, інші елементи наслідують його.

25) Для елемента `<body>` задана властивість `color` з домінантним на макеті кольором тексту. Решта тексту наслідує або перекриває це значення.

26) Розмір шрифту (властивість `font-size`) всіх текстових елементів точно відповідає значенням з макета.

27) Висота рядка (властивість `line-height`) всіх текстових елементів точно відповідає значенням з макета і задана як множник, а не в `px`.

28) Колір (властивості `color` та `background-color`) всіх елементів точно відповідає значенням з макета.

29) Вага шрифту (властивість `font-weight`) всіх текстових елементів точно відповідає значенням з макета. Вага шрифту явно вказана тільки, якщо значення в макеті відрізняється від стандартного для цього елемента в браузері.

30) Кнопкам задана властивість `cursor` зі значенням `pointer`.

31) У стилях не повторюються значення властивостей, які задані браузером за замовчуванням. Наприклад, посиланням не потрібно вказувати `cursor: pointer`, а абзацам — `font-style: normal` або `font-weight: 400`.

2. Нормалізація та скидання стилів в розгорнутому проєкті

Виконаний проєкт на даному етапі має відповідати таким

вимогам:

- 1) Підключений нормалізатор стилів [modern-normalize](#).
- 2) Підключений [reset.css](#) на сторінку. Проаналізувати моменти, які змінились.
- 3) Перевірити наявність властивості [box-sizing: border-box](#) для елементів.
- 4) Усі кольори, присутні на макеті, винесені у CSS-змінні.

Заняття 5. Flex-box, відступи, контейнер.

Актуальність та цілі заняття

Заняття 5. «Розміщення елементів сторінки

через технологію flex-box, відступи, контейнер. Тренування з викладачем»

ХІД ЗАНЯТТЯ

На цьому занятті ми зосередимося на практичних аспектах розміщення елементів сторінки відповідно до дизайн-макета. Після створення семантичної структури сторінки важливо навчитися правильно організовувати контент у межах секцій, керувати відступами та вирівнюванням елементів.

Під час заняття розглянемо принцип роботи контейнера для центрування та обмеження контенту по ширині, навчимося вимірювати його розміри в макеті та стилізувати в CSS. Також пригадаємо й закріпимо використання технології Flexbox для розміщення елементів усередині секцій згідно з макетом.

Отримані знання дозволять коректно організовувати структуру сторінки, зберігати цілісність дизайну та підготувати основу для подальшої роботи над проєктом.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Щоб контент коректно відображався на екранах будь-якого розміру та завжди був вирівняний по центру, рекомендується використовувати контейнер для центрування вмісту сайту. Для цього в контейнері для розміщення елементів згідно з макетом сьогодні використовують технологію flex-box.

ЦІЛІ ЗАНЯТТЯ

Пригадати властивості flex-box, як вимірювати контейнер для центрування контенту сайту та стилізувати його.

ОСНОВНІ ЗАВДАННЯ

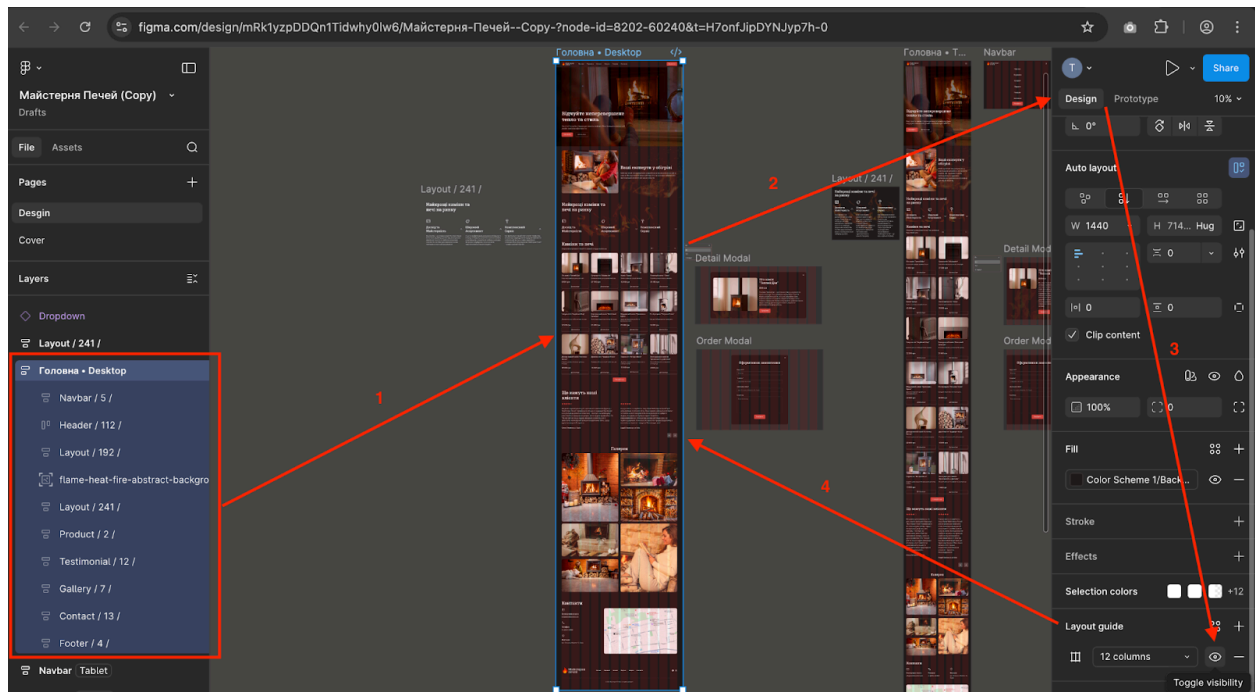
1. Навчитися вимірювати `.container` на макеті та стилізувати його.
2. Розібратися як правильно розміщувати секції та відсувати контент всередині них.
3. Розмістити всі елементи секцій згідно [з макетом](#), використовуючи властивості flex-box.

Підготовка до заняття

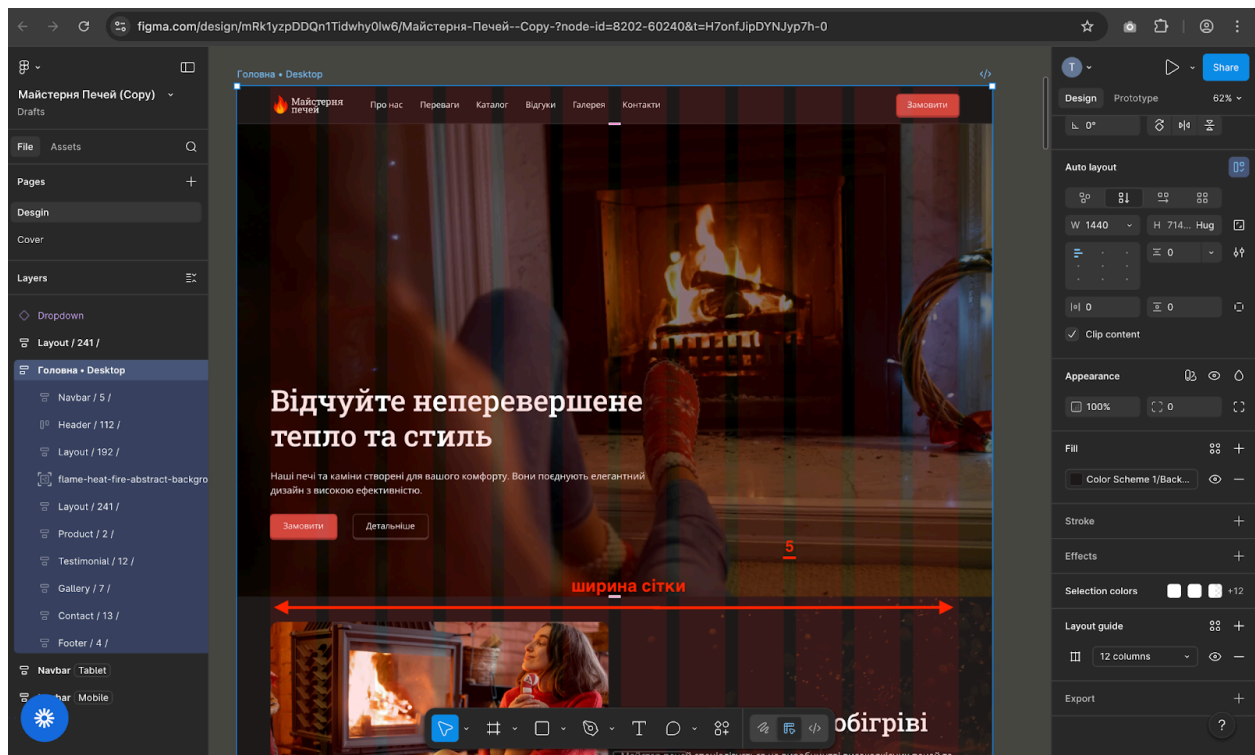
1. Контейнер для вирівнювання контенту сторінки

Контейнер, обгортка, враппер, центрувальник — все це імена стандартного блокового елемента-контейнера для обмеження ширини та центрування контенту сторінки.

Щоб поміряти контейнер, потрібно у figma зробити копію оригінального макета, далі у копії виділити сторінку макета (1) та праворуч у вкладці Design (2), відкрити іконку ока на меню Layout grids (3):



Як тільки сітка стала видимою у вигляді червоних стовпців, міряємо її (4):



Червоні стовпчики показують ширину сітки. Ширина контейнера складається з двох частин:

- ширини сітки — 1312px;
- відступів (padding) (5) зліва і справа, щоб контент не торкався країв екрана.

Щоб визначити padding, потрібно виміряти відстань між червоними стовпчиками — вона дорівнює 24px.

Цю відстань ділимо навпіл: $24\text{px} / 2 = 12\text{px}$ — це padding зліва і справа.

Отже, фінальна ширина контейнера: $1312\text{px} + 24\text{px} = 1336\text{px}$.

В CSS створюється клас `container`, який можна задати будь-якому елементу, але на практиці це завжди буде несемантичний `<div>`.

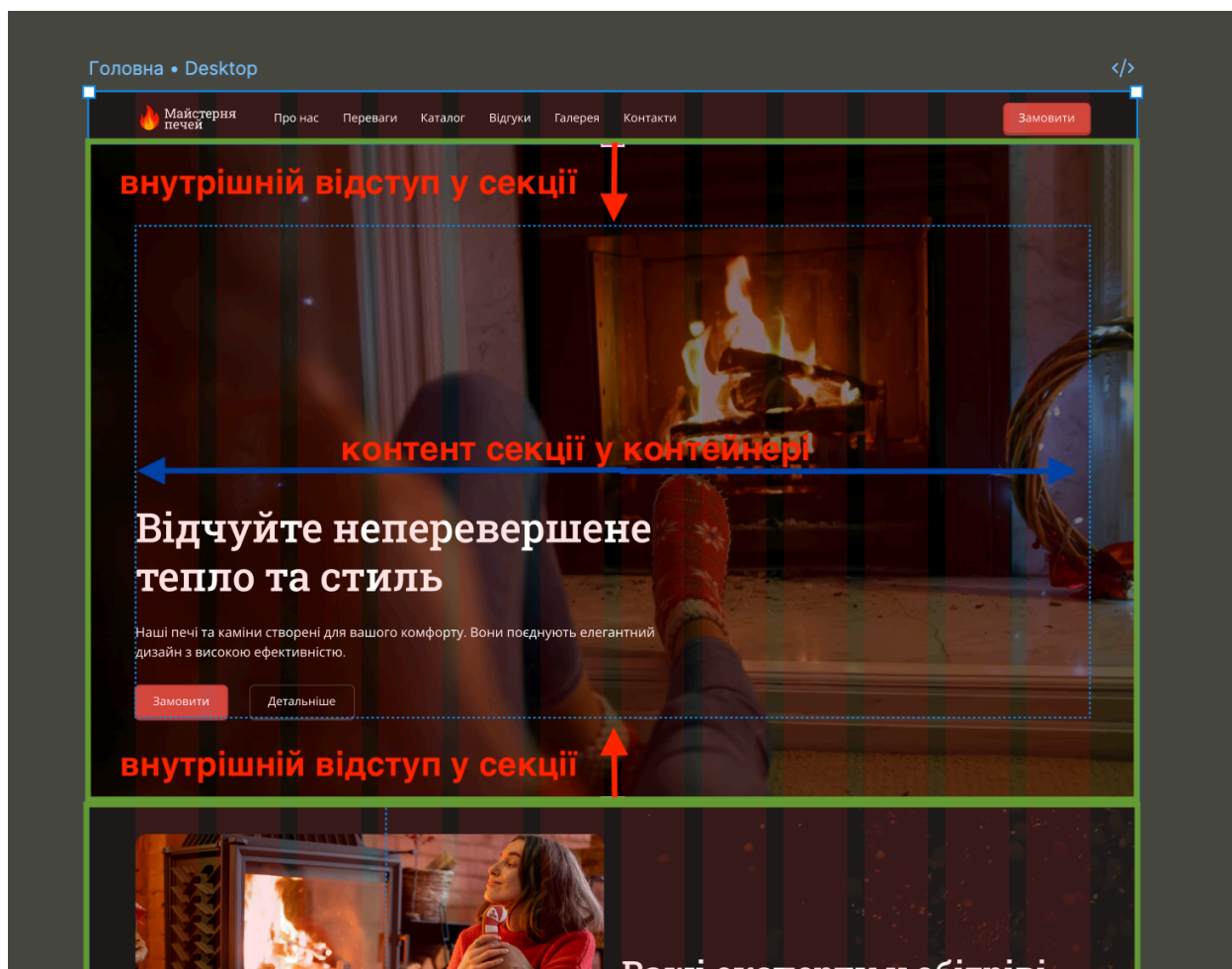
Контейнеру обов'язково встановлюється ширина (фіксована або адаптивна), відповідно до сітки з макета, автоматичні горизонтальні відступи й необов'язкові поля, також за макетом:

```
styles.css
.container {
  width: 1336px;
  margin: 0 auto;
  padding: 0 12px;
}
```

Контейнер може бути всього один — загальний, який обгортає відразу увесь контент сторінки. У [прикладі](#) продемонстровано межі контейнера виділені пунктиром для наочності. Найчастіше контейнерів декілька, оскільки в кожному розділі (секції) сторінки необхідний свій контейнер. За такого підходу можна зручно задавати різний повнорозмірний фон та інші стилі оформлення окремих розділів сторінки.

2. Розміщення секцій та відсування контенту всередині них

Хорошою практикою є розміщення секцій таким чином, щоб вони прилягали одна до одної, без відступів між ними. Проте, всередині секції варто відсунути внутрішній контент від верхньої та нижньої границь.



Орієнтуючись на [макет](#) потрібно поміряти та додати внутрішні відступи у секціях, які відсувають дочірні елементи від верхньої та нижньої границь секцій. Всі секції повинні прилягати одна до одної, без відступів. Такий підхід дозволить зберігати гарну заливку фону у секції.

3. Розміщення елементів у секції

Розпочинаємо працювати над розміщенням елементів у секціях за обраним [макетом](#). Для цього спершу варто пригадати властивості flex-box на чудовому тренажері [flexboxfroggy](#).

А також обов'язково проглянути **вимоги прийняття проєкту** на даному етапі:

- 1) У корені проєкту є папка **images** з зображеннями.
- 2) У корені проєкту є папка **CSS** з файлом стилів.
- 3) Всі стилі написані в одному файлі **styles.css**, який знаходиться в папці **CSS**.
- 4) У назвах файлів відсутні великі літери, пробіли й трансліт. Назви містять тільки літери та слова англійської мови.
- 5) Вихідний код відформатований за допомогою **Prettier**.
- 6) Всі зображення та текстовий контент взяті з макета.
- 7) Підключений нормалізатор стилів [modern-normalize](#).
- 8) Код написаний з дотриманням [настанови](#).
- 9) Допускається глобальне скидання стилів за селектором тегу для елементів **<h1>...<h6>**, **<p>** і ****.
- 10) В елементів відсутні зовнішні відступи (властивість **margin**) що «пробивають» батьківський елемент.

- 11) Для вертикальних відступів між двома сусідніми елементами використовується властивість `margin`.
- 12) Для проміжку між рамкою батька і його дитиною використовується властивість `padding`.
- 13) Розміри зовнішніх відступів (властивість `margin`) і внутрішніх полів (властивість `padding`) елементів задані точно за макетом.
- 14) Створений загальний допоміжний клас `.container` для центрування та обмеження контенту по ширині.
- 15) Ширина «контейнера» відповідає макету і дорівнює `1336px`.
- 16) «Контейнер» обгортає контент хедера, футера і секцій. Тобто знаходиться всередині них.
- 17) Для розташування елементів використовується `Flexbox`, але тільки там, де це необхідно. Наприклад у шапці, навігації, списках в секціях тощо, тобто там, де розташувати елементи горизонтально по-іншому неможливо.
- 18) Фінальні розміри блоків в браузері відповідають макету.
- 19) В елементів відсутня фіксована висота, вона визначається їх контентом.
- 20) У хедера є нижня рамка, необхідно сильно наблизити макет, щоб побачити її.

21) Секції розташовані одна під одною як стопка книг, без зовнішніх відступів.

22) Для більшості секцій використовується клас `.section`, якому задані верхні та нижні падінги по `48px`, що відсувають контент всередину секції.



Для детальнішого повторення матеріалів попередніх дисциплін, необхідних для виконання даного етапу проєкту, рекомендуємо ознайомитись із темами 8-12 дисципліни «Основи веброзробки (мови розмітки)»:

[Тема 8. Flexbox](#)

[Тема 9. CSS Grid](#)

[Тема 10. Властивості контейнера](#)

[Тема 11. Робота з властивостями контейнера, ч.1](#)

[Тема 12. Робота з властивостями контейнера, ч.2](#)

Підсумки

Отже, сьогодні ми виконали найцікавішу частину розробки сайту – розміщення контенту в порядку згідно [з макетом](#). Чудово попрацювали! Давайте закріпимо наші здобутки на цьому занятті:

Ми пригадали, що таке контейнер оскільки його виміряти, щоб розміщення елементів точно відповідало дизайн-макету.

Попрактикувалися у стилізації контейнера та розміщення елементів у контейнері за допомогою CSS-властивостей, використовуючи технологію flex-box.

Наступного разу на практиці ми доробимо розміщення елементів у всіх секціях та далі зосередимось на проєкті [Flora](#): перевіримо коректність виміряного вами контейнеру, розміщення всіх елементів у ньому та відступів. До зустрічі!

Заняття 6. Flex-box, відступи, контейнер.

Актуальність та цілі заняття

Заняття 6. «Розміщення елементів сторінки через технологію flex-box, відступи, контейнер. Практика»

ХІД ЗАНЯТТЯ

Доброго дня! На попередньому занятті ми ознайомилися з теорією: дізналися про контейнер, центрування контенту та основи роботи з Flexbox.

Сьогодні час застосувати ці знання на практиці й побачити реальний результат у проєкті Flora. Це буде заняття максимальної концентрації, присвячене розміщенню елементів у секціях, налаштуванню відступів та правильному позиціонуванню контенту згідно з макетом.

Ми перевіримо: чи правильно виміряний контейнер, чи коректно розташовані елементи в секціях, і чи відповідає верстка всім вимогам макета. Готуй свій проєкт — сьогодні ми закріплюємо навички роботи з Flexbox і контейнерами, щоб твоя верстка була точною та професійною!

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Ми з вами вже досить далеко пройшли й багато чого вміємо. Але сьогодні ми навчимося думати на декілька кроків вперед та верстати сайти так, щоб на їхню адаптацію довелося витратити мінімум часу. В цьому нам допоможе технологія Flex-box.

ЦІЛІ ЗАНЯТТЯ

Потренуватися правильно додавати відступи між елементами у секціях та розміщати їх згідно [з макетом](#).

ОСНОВНІ ЗАВДАННЯ

1. Закріпити навички вимірювання контейнера на дизайн-макеті.
2. У [проєкті](#) правильно розмістити всі елементи у секціях.

Підготовка до заняття

1. Створення контейнера та розміщення елементів у секціях згідно з макетом

Для закріплення на практиці виконай такі завдання:

В репозиторії [UMT-markup-practice-ПІБ](#) помірай контейнер сторінки з [макету](#) та добавь його у своєму проєкті.

Додай стилі для геометрії (ширину, відступи, поля і рамки) і позиціонування контенту за допомогою Flexbox для сторінок згідно [з макетом](#).

Налаштуй GitHub Pages і добавь посилання на живу сторінку в секцію [About](#) GitHub-репозиторію.

Виконаний проєкт на даному етапі має відповідати таким **вимогам**:

- 1) У корені проєкту є папка [images](#) з зображеннями.
 - 2) У корені проєкту є папка [css](#) з файлом стилів.
 - 3) Всі стилі написані в одному файлі [styles.css](#), який знаходиться в папці [css](#).
 - 4) У назвах файлів відсутні великі літери, пробіли й трансліт. Назви містять тільки літери та слова англійської мови.
 - 5) Вихідний код відформатований за допомогою [Prettier](#).
 - 6) Всі зображення та текстовий контент взяті з макета.
 - 7) Підключений нормалізатор стилів [modern-normalize](#).
 - 8) Код написаний з дотриманням [настанови](#).
- Допускається глобальне скидання стилів за селектором тегу для елементів [<h1>...<h6>](#), [<p>](#) і [](#).
- 10) В елементів відсутні зовнішні відступи (властивість [margin](#)) що «пробивають» батьківський елемент.
 - 11) Для вертикальних відступів між двома сусідніми елементами використовується властивість [margin](#).
 - 12) Для проміжку між рамкою батька і його дитиною використовується властивість [padding](#).

- 13) Розміри зовнішніх відступів (властивість `margin`) і внутрішніх полів (властивість `padding`) елементів задані точно за макетом.
- 14) Створений загальний допоміжний клас `.container` для центрування та обмеження контенту по ширині.
- 15) Ширина «контейнера» відповідає макету і дорівнює `1336px`.
- 16) «Контейнер» обгортає контент хедера, футера і секцій. Тобто знаходиться всередині них.
- 17) Для розташування елементів використовується `Flexbox`, але тільки там, де це необхідно. Наприклад у шапці, навігації, списках в секціях тощо, тобто там, де розташувати елементи горизонтально по-іншому неможливо.
- 18) Фінальні розміри блоків в браузері відповідають макету.
- 19) В елементів відсутня фіксована висота, вона визначається їх контентом.
- 20) У хедера є нижня рамка, необхідно сильно наблизити макет, щоб побачити її.
- 21) Секції розташовані одна під одною як стопка книг, без зовнішніх відступів.
- 22) Для більшості секцій використовується клас `.section`, якому задані верхні та нижні падінги по `48px`, що відсувають контент всередину секції.

Заняття 7. «Декори, анімації та бібліотеки.

Актуальність та цілі заняття

Заняття 7. «Декори, анімації та бібліотеки.

Тренування з викладачем»

ХІД ЗАНЯТТЯ

На цьому занятті ми перейдемо від структурної розмітки до декоративного оформлення сторінки. Після створення семантичного каркаса важливо навчитися правильно працювати з фоновими та декоративними зображеннями, багат шаровими фонами, градієнтами й тінями, а також застосовувати анімації для інтерактивності.

Сьогодні ми зосередимося на практичному використанні контентних і декоративних елементів, підключенні векторної графіки та реалізації ефектів згідно з макетом Flora. Це заняття присвячене тренуванню навичок, які дозволяють створювати естетично приємні та професійно виконані інтерфейси.

Ми перевіримо: чи правильно додані фони та декоративні елементи, чи коректно підключені SVG-іконки, і чи працюють CSS-переходи та анімації відповідно до макета. Готуй свій проєкт — сьогодні закріплюємо навички роботи з декорами та анімаціями на практиці!

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Важливо розібратися як правильно користуватися зображеннями та декорами на практиці в роботі зі стилями згідно з макетом, щоб створювати дивовижні ефекти, розвивати свій рівень професійності. Для того щоб під час верстки правильно працювати із зображеннями та декорами, необхідно розрізняти типи зображень — контентні та декоративні, їх відмінності та особливості застосування, ефектів. Фони, декорації та анімації — це не просто «красивості»: вони:

- підсилюють візуальну ієрархію;
- полегшують сприйняття інформації;
- підвищують UX-якість взаємодії;
- роблять інтерфейс живим і сучасним.

ЦІЛІ ЗАНЯТТЯ

Відпрацювати навички використання фонів, декоративних елементів і анімацій відповідно до заданого макета.

ОСНОВНІ ЗАВДАННЯ

1. Навчитись застосовувати контентні та декоративні зображення, багат шарові фони та градієнти, тіні, створювати декоративні ефекти згідно з макетами.
2. Пригадати графічні формати зображень, що таке векторна графіка і SVG теги, і чим вона відрізняється від растрової.

3. Потренувати навички застосування векторних зображень, збирати їх в єдиний файл через сервіс [iconmoon](#) та підключати до проєкту.
4. Згадати для чого потрібні CSS-переходи та навчитись застосовувати властивості [transition](#) & [transform](#).
5. Реалізувати фони, векторну графіку та інші декори згідно [з макетом](#).

Підготовка до заняття

1. Фон елемента

Фон — це візуальне заповнення області блоку (секції, контейнера або елемента), що може бути:

- однотонним (колір);

- зображенням;

- градієнтом;

- багатошаровим поєднанням зображення + градієнт + колір.

Фонові зображення допомагають:

- виділити секцію;

- створити потрібну атмосферу;

- підкреслити контент та композицію макета.

При роботі над проєктом за [макетом](#) необхідно:

1) Перевірити макет на предмет фонових зображень (широкі блоки, заголовні секції).

2) Додати фонові зображення через CSS властивість:

```
.section {  
  
    background-image: url('../images/bg-hero.jpg');  
  
    background-size: cover;  
  
    background-position: center;  
  
}
```

3) Якщо у макеті фон із затемненням або градієнтом — використати багат шарові фони:

```
Background-image:  
  
linear-gradient(rgba(0,0,0,0.4), rgba(0,0,0,0.4)),  
url('../images/bg.jpg');
```

Це дозволяє затемнити фон для кращого читання тексту.

4) Переконайтеся, що фон не виходить за межі контенту і не пікселюється на великих екранах.

Підсумки

Отже, сьогодні ми попрактикувались в роботі з контентними та декоративними зображеннями, багатошаровими фонами, градієнтами та тінями, створили декоративні ефекти згідно з макетами. Тож підсумуємо наші здобутки на цьому занятті:

Ми пригадали, як працювати з фоновими зображеннями та багатошаровими фонами.

Попрактикувалися в роботі з векторними зображеннями, створили декоративні ефекти, використовуючи svg-формат для зображень.

Додали анімації та пофантазували з анімаційними ефектами.

Наступного разу ми перейдемо до практики на проєкті [Flora](#): перевіримо коректність вашої роботи з векторною графікою та анімаціями.

До зустрічі!

Заняття 8. Декори, анімації та бібліотеки

Актуальність та цілі заняття

Заняття 8. «Декори, анімації та бібліотеки.

Практика»

ХІД ЗАНЯТТЯ

Доброго дня!

На попередніх заняттях ми опрацювали теорію роботи з фоновими та декоративними зображеннями: розібралися з багат шаровими фонами, градієнтами, тінями, а також із використанням CSS-анімацій і переходів для створення інтерактивності.

Сьогоднішнє заняття повністю практичне. Ми застосуємо отримані знання у проєкті Flora та детально попрацюємо з вашим кодом. Основний фокус — коректне використання декорів і анімацій відповідно до дизайн-макета та вимог до професійної верстки.

Під час заняття ми перевіримо:

- як додані фонові та декоративні зображення;
- чи правильно підключені й використані SVG-іконки;
- чи відповідають CSS-переходи та анімації макета і технічним вимогам.

Готуй свій проєкт до роботи — сьогодні ми закріплюємо навички, які напряду впливають на якість верстки та відрізняють навчальний проєкт від продакшн-рішення.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Практична робота з декорами та анімаціями є важливим компонентом підготовки фронтенд-розробника. Саме на цьому етапі формується вміння перетворювати дизайн-макет на живий, інтерактивний інтерфейс, який відповідає вимогам UX та сучасних вебстандартів. Фони, декоративні елементи, SVG-іконки та анімації — це не лише візуальна складова, а й показник якості реалізації проєкту. Вони впливають на:

- сприйняття інтерфейсу користувачем;
- плавність взаємодії та відчуття «живого» продукту;
- відповідність роботи реальним продакшн-вимогам.

ЦІЛІ ЗАНЯТТЯ

Сформувати впевнені навички роботи з фонами, декором і анімаціями на основі реального дизайн-макета. У результаті практики ви:

- навчитесь правильно інтерпретувати дизайн;
- зменшите кількість типових помилок у роботі з декоративними елементами;
- закріпите навички, релевантні вимоги code review та технічної оцінки проєктів.

ОСНОВНІ ЗАВДАННЯ

1. Оцінити [макет](#) і виявити всі декоративні елементи.
2. Перевірити зроблену частину проєкту та закріпити навички додавання декорів за дизайн-макетом.
3. Додати фонові стилі, градієнти, тіні.
4. Підключити та стилізувати SVG-іконки.
5. Реалізувати CSS-анімації для інтерактивності.
6. Забезпечити плавні переходи та ефекти згідно з макетом.
7. Використати бібліотеки для анімацій.

Підготовка до заняття

1. Додавання декорів на сайт

Для закріплення отриманих знань на практиці виконай такі завдання:

В репозиторії `UMT-markup-practice-ПІБ` додай розмітку та оформлення іконок і декоративних ефектів з [макету](#).

Для генерації SVG-спрайту використовуй сервіс [Icomoon](#).

Для оптимізації створеного SVG-спрайту використовуй сервіс [svgomg](#).

Налаштуй `GitHub Pages` і додай посилання на живу сторінку в секцію `About` `GitHub`-репозиторію.

Виконаний проєкт на даному етапі має відповідати таким **вимогам**:

- 1) У корені проєкту є папка `images` з зображеннями.

- 2) Усі векторні зображення (іконки) зібрані у SVG-спрайт `icons.svg`, який лежить у папці `images`.
- 3) Усі векторні зображення оптимізовані.
- 4) У корені проєкту є папка `css` з файлами стилів.
- 5) Всі стилі написані в одному файлі `styles.css`, який знаходиться в папці `css`.
- 6) У назвах файлів відсутні великі літери, пробіли та трансліт. Назви містять тільки літери й слова англійської мови.
- 7) Вихідний код відформатований за допомогою `Prettier`.
- 8) Всі зображення та текстовий контент взяті з макета.
- 9) Підключений нормалізатор стилів [modern-normalize](#)
- 10) Всі стилі написані в одному файлі `styles.css`, який знаходиться в папці `css`.
- 11) Код написаний з дотриманням [настанови](#).
- 12) Для всіх іконок використовується векторна графіка у форматі `svg`.
- 13) SVG-іконки експортовані правильно. При експорті обрана «група», а не сам вектор.
- 14) Всі іконки з SVG-спрайту додані в HTML за допомогою тегів `<svg>` і `<use>`.

- 15) В секції «Get in Touch», футері та у кнопках слайдерів додані іконки.
- 16) В секції з відгуками додані іконки-зірочки.
- 17) В секції з контактами додані іконки.
- 18) У футері додані іконки соцмереж.
- 19) Виконана HTML-розмітка всіх елементів макета.
- 20) Теги використані відповідно до їх семантичного змісту.
- 21) Велике зображення з ефектом затемнення (під хедером) виконано як фон. Для затемнення використовується багат шаровий фон з градієнтом.
- 22) Фонове зображення в блоці під хедером не розтягується ширше свого оригінального розміру 1440px.
- 23) При ховері або фокусі іконки повинні переходити в активний стан — змінювати колір, якщо це зазначено в макеті.
- 24) Для всіх ефектів ховера і фокуса (колір, фон, тінь) зроблені переходи. Час — 250ms, функція розподілу часу — cubic-bezier(0.4, 0, 0.2, 1).
- 25) У переходах явно зазначені анімовані властивості. Ніде немає значення all.

Заняття 9. Адаптив сторінки за принципом

Актуальність та цілі заняття

Заняття 9. «Адаптив сторінки за mobile-first.

Тренування з викладачем»

ХІД ЗАНЯТТЯ

Ми вже навчилися додавати інтерфейсу виразності за допомогою декорів, анімацій та готових бібліотек. Тепер настав час зробити наш проєкт по-справжньому зручним для користувача на будь-якому пристрої.

Сьогодні ми зосередимося на одному з ключових підходів сучасної верстки — **mobile-first** та навчимося будувати адаптивну сторінку, починаючи з мобільної версії й поступово розширюючи її для планшетів і десктопів. Ви попрактикуєтеся працювати з брейкпоінтами, медіазапитами та гнучкими макетами, щоб інтерфейс коректно виглядав на різних розмірах екранів.

Адаптивність — це не лише про вигляд, а й про логіку побудови стилів, продуктивність і зручність підтримки коду. Саме тому під час тренування з викладачем ви закріпите навички створення масштабованої структури стилів і підготуєте проєкт до реальних вимог сучасної веброзробки. Починаймо!

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Більшість користувачів сьогодні взаємодіють із вебсайтами з мобільних пристроїв, тому підхід mobile-first є базовим стандартом сучасної фронтенд-розробки. Саме від того, наскільки правильно спроектована мобільна версія сторінки, залежить зручність користування, швидкість завантаження та подальша масштабованість інтерфейсу.

ЦІЛІ ЗАНЯТТЯ

Навчитися реалізовувати адаптивні макети, логічно вибудовувати систему брейкпоінтів і застосовувати медіазапити відповідно до реальних вимог індустрії та макету.

ОСНОВНІ ЗАВДАННЯ

1. Реалізувати базову мобільну та десктопну версії сайту згідно [з макетом](#), поступово адаптувати її для більших екранів.
2. Навчитись коректно використовувати медіазапити за вказаними брейкпоінтами.
3. Потренуватись перевіряти та налагоджувати відображення сторінки на різних розмірах екранів за допомогою інструментів розробника.
4. Перевірити реалізований код та виправити допущені помилки.

Підготовка до заняття

1. Підхід mobile-first

Mobile-first — це підхід, при якому стилі для мобільних пристроїв задаються першими, а адаптація для більших екранів робиться поступово за допомогою медіазапитів. Головна ідея: починати з простих та базових стилів для найменших екранів і масштабувати дизайн для планшетів та десктопів.

Спочатку задаються базові стилі для мобільної версії (найменший екран), а потім за допомогою `@media (min-width: ...)` додаються стилі для більших екранів.

Медіа-запити бувають:

`min-width` — застосовує стилі, коли ширина екрана **не менша** за вказане значення (часто використовується у mobile-first).

`max-width` — застосовує стилі, коли ширина екрана **не більша** за вказане значення (часто використовується у desktop-first).

CSS читається зверху вниз. У mobile-first базові стилі для мобільних пристроїв пишуться спершу, а стилі для більших екранів через медіазапити дописуються нижче. Каскад забезпечує, що стилі для більшого екрана замінюють базові.

В своєму макеті використовуй також властивості, які допомагають зробити елементи гнучкими:

`width: 100%` і `max-width` для блоків і зображень;
`flex-grow`, `flex-shrink`, `flex-wrap` для flex-контейнерів;
`grid-template-columns: repeat(auto-fit, minmax(...))` для grid.

2. Адаптив сторінки за mobile-first

Щоб правильно організувати структуру CSS для адаптиву, необхідно:

Спершу виконати базові стилі для мобільної версії (без медіазапитів).

Перевірити чи доданий viewport meta tag, тобто `<meta name="viewport" content="width=device-width, initial-scale=1.0">`

Додати стилі для планшетів через `@media (min-width: ...)`.

Далі додаємо стилі для десктопів через `@media (min-width: ...)`. Це забезпечує логічну, підтримувану та масштабовану структуру.

Breakpoints — це точки, при яких макет або дизайн змінюється під нову ширину екрана. Обирають їх, виходячи з логіки дизайну, а не конкретних пристроїв, наприклад: 480px, 768px, 1024px, 1200px.

Перейди за посиланням на [макет](#), за яким будемо адаптувати сайт та проаналізуй детальніше його брейкпоінти та [вимоги](#) до його розробки, а також **критерії приймання проєкту**:

- 1) При перегляді сторінки на будь-якому пристрої шириною від 320px, не з'являється горизонтальна смуга прокручування.
- 2) Вихідний код відформатований за допомогою Prettier.
- 3) Всі зображення та текстовий контент взяті з макета.
- 4) Підключений нормалізатор стилів [modern-normalize](#).

- 5) Код написаний з дотриманням [настанови](#).
- 6) У всіх сторінок в блоці `<head>` є метатег `viewport`.
- 7) Верстка виконана щодо трьох точок перелому: `375px`, `768px` і `1440px`.
- 8) При написанні стилів використаний `Mobile First` підхід і медіа-функція `min-width`.
- 9) Стили, необхідні тільки у певному проміжку, закриті в медіа-запити `(min-width: ) and (max-width: )` або тільки `(max-width: )`.
- 10) В медіа-запитах відсутнє зайве дублювання стилів.

Щоб **перевіряти брейкпоінти** та різні розміри екранів у DevTools, необхідно:

у Chrome/Edge відкрийте DevTools → натисніть іконку Toggle device toolbar або Ctrl+Shift+M;
обери потрібний пристрій або задай власну ширину екрана;
перевіряй, як змінюється верстка та стилі при проходженні через різні брейкпоінти.

Для детальнішого повторення матеріалів попередніх дисциплін, необхідних для виконання даного етапу проєкту, рекомендуємо ознайомитись із темою 17 «[Адаптивний дизайн](#)» дисципліни «Основи веброзробки (мови розмітки)».

Заняття 10. Адаптив сторінки за принципом

Актуальність та цілі заняття

Заняття 10. «Адаптив сторінки за mobile-first.

Практика»

ХІД ЗАНЯТТЯ

На цьому занятті ми переходимо від теорії та тренувань до **повноцінної практики**: адаптуємо сторінки та секції проєкту **Flora** під різні екрани, закріплюючи всі навички, отримані під час попереднього заняття. Ви будете працювати з медіазапитами, breakpoints, гнучкими блоками та перевіряти адаптивність у DevTools.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Вміння правильно адаптувати сторінку до різних екранів без горизонтальної прокрутки, дублювання стилів та порушення макета — основа сучасної фронтенд-розробки. Це практичне заняття допоможе навчитися будувати масштабовані та підтримувані інтерфейси, що коректно працюють на мобільних, планшетах та десктопах.

ЦІЛІ ЗАНЯТТЯ

1) Закріпити навички адаптивної верстки за принципом mobile-first.

- 2) Навчитися працювати з реальним макетом Flora, застосовуючи медіазапити та breakpoints.
- 3) Перевіряти та налагоджувати адаптивний код у DevTools.

ОСНОВНІ ЗАВДАННЯ

1. Адаптувати сторінки та секції сайту згідно [з макетом](#) Flora.
2. Перевірити правильність застосування медіазапитів та breakpoints у проєкті [макету](#) Flora.
3. Забезпечити гнучке масштабування блоків, зображень та контенту.

Підготовка до заняття

Підготовка до заняття

Переглянь попередні матеріали з mobile-first та адаптивної верстки, а саме:

принцип побудови базових стилів для мобільних екранів;
використання `@media (min-width: ...)`;
налаштування viewport meta tag;
гнучкі блоки (flexbox, grid) та масштабування зображень.

На даному етапі виконання свого проєкту **необхідно реалізувати такі критерії прийняття:**

- 1) При перегляді сторінки на будь-якому пристрої шириною від `320px`, не з'являється горизонтальна смуга прокручування.

- 2) Вихідний код відформатований за допомогою `Prettier`.
- 3) Всі зображення та текстовий контент взяті з макета.
- 4) Підключений нормалізатор стилів [modern-normalize](#).
- 5) Код написаний з дотриманням [настанови](#).
- 6) У всіх сторінок в блоці `<head>` є метатег `viewport`.
- 7) Верстка виконана щодо трьох точок перелому: `375px`, `768px` і `1440px`.
- 8) При написанні стилів використаний `Mobile First` підхід і медіафункція `min-width`.
- 9) Стили, необхідні тільки у певному проміжку, закриті в медіазапит `(min-width: ) and (max-width: )` або тільки `(max-width: )`.
- 10) В медіазапитах відсутнє зайве дублювання стилів.

Перевіряй адаптивність у DevTools:

- у *Chrome/Edge* відкрий *DevTools* → *Toggle device toolbar* (*Ctrl+Shift+M*);
- обери потрібний пристрій або задай власну ширину;
- сліdkуй, як змінюється макет при проходженні через різні брейкпоінти.

Заняття 11. Адаптив сторінки mobile-first.

Актуальність та цілі заняття

Заняття II. «Адаптив сторінки за принципом

mobile-first. Мобільне меню. Тренування з викладачем»

ХІД ЗАНЯТТЯ

Сьогодні ми зосередимося на одному з ключових елементів мобільного інтерфейсу — **мобільному меню**. Навчимося правильно створювати розмітку, стилізувати та реалізовувати його відкриття/закриття для мобільних пристроїв. Ви попрактикуєтеся застосовувати **mobile-first підхід**, використовувати медіазапити та класи для управління станом меню.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Мобільне меню — важливий елемент зручності користування. Правильно реалізоване меню забезпечує логічну навігацію, швидкий доступ до всіх секцій сайту та підвищує юзабіліті на будь-якому пристрої. Знання, як зробити адаптивне меню, є обов'язковим для сучасних фронтенд-розробників.

ЦІЛІ ЗАНЯТТЯ

- 1) Навчитися створювати мобільне меню за принципом mobile-first.
- 2) Закріпити навички адаптивної верстки з медіазапитами та гнучкими блоками.
- 3) Навчитися контролювати відкриття та закриття меню через класи та CSS-переходи.

ОСНОВНІ ЗАВДАННЯ

1. Навчитись робити базову розмітку мобільного меню згідно макету.
2. Реалізувати стилізацію меню та його елементів для мобільних екранів.
3. Забезпечити правильну поведінку меню при додаванні класу `is-open`.

Підготовка до заняття

1. Розмітка мобільного меню

Спершу всі елементи мобільного меню потрібно **додати до HTML-документу** згідно макету. Контейнер меню повинен містити список пунктів меню, а також кнопку-бургер для відкриття та закриття меню. Початкове відображення меню має бути прихованим, що створює базову основу для подальшої взаємодії.

Всі елементи, включно з контейнером меню, списком пунктів та кнопкою-бургером для відкриття і закриття, мають бути додані у відповідності до макету.

Для пунктів меню рекомендується **використовувати семантичну структуру `<ul>` та `<li>`**, що підвищує доступність і полегшує навігацію користувачам, а також допомагає сканерам пошукових систем розпізнати структуру контенту.

Крім того, слід перевірити наявність атрибутів `aria` для елементів управління (наприклад, `aria-expanded` у кнопки-бургера), що покращує доступність меню для людей з обмеженими можливостями.

2. Стилiзацiя мобiльного меню

Наступним кроком є **пiдготовка стилiв для мобiльного меню**. На етапi пiдготовки слiд ознайомитися з особливостями стилiзацiї мобiльного меню. Меню має займати всю висоту viewport (`height: 100vh`) і вiдповiдати ширинi, передбаченiй макетом, щоб воно виглядало пропорцiйно на всiх мобiльних пристроях.

Важливо **стилiзувати всi елементи меню**: пункти списку, кнопки, iконки та фоновi блоки, враховуючи кольорову палiтру, розмiри шрифтиv та мiжрядковi iнтервали макету.

Кожен пункт меню, кнопка та додаткові елементи інтерфейсу повинні бути **стилізовані з урахуванням гнучких блоків** (**flex** або **grid**), щоб забезпечити правильне розташування і адаптивність для різних екранів та забезпечити правильну поведінку при зміні розміру екрану. Особливу увагу варто приділити відступам і міжрядковим інтервалам, щоб меню залишалося зручним для користувача навіть на малих екранах.

3. Логіка відкриття та закриття меню

Для коректної роботи мобільного меню необхідно розуміти, як реалізувати його динамічну поведінку, опрацювати **механізм його відкриття та закриття**. Початково меню приховане, і його відображення реалізується через додавання класу **is-open** на контейнер меню. Додатково слід перевірити поведінку елементів при відкритті та закритті, а також, при бажанні, застосувати CSS-транзиції або анімації для плавного відображення.

Щоб перевірити взаємодію користувача з кнопкою-бургером, при натисканні меню має відкриватися, а при повторному натисканні або натисканні на фон — закриватися. Додатково рекомендується налаштувати логіку, щоб меню закривалося при виборі пункту списку, якщо це відповідає UX-макету. Також слід перевірити коректну роботу атрибутів **aria**, щоб зміни стану меню

були доступні для допоміжних технологій, забезпечуючи доступність інтерфейсу для всіх користувачів.

4. Mobile-first, медіазапити та перевірка адаптивності

Стилі для мобільного меню необхідно писати з урахуванням підходу mobile-first. Це означає, що базові стилі створюються спочатку для найменших екранів, а для більших екранів додаються через медіа-запити `@media (min-width: ...)`. Під час підготовки варто перевірити, щоб медіа-запити не містили дублювання стилів і щоб структура CSS залишалася логічною та підтримуваною.

Важливим етапом роботи над практичним завданням є **перевірка адаптивності мобільного меню у браузерних інструментах розробника**. Необхідно переконатися, що меню коректно відкривається та закривається на різних ширинах екранів, не виходить за межі viewport і відповідає макету. Також варто протестувати поведінку меню на різних точках перелому (breakpoints), щоб упевнитися в правильності його адаптивної верстки.

Перейди за посиланням на [макет](#), з яким будемо працювати та проаналізуй детальніше його і [вимоги](#) до його розробки, а також **критерії прийняття проєкту**:

- 1) При перегляді сторінки на будь-якому пристрої шириною від 320px, не з'являється горизонтальна смуга прокручування.
- 2) Вихідний код відформатований за допомогою Prettier.
- 3) Всі зображення та текстовий контент взяті з макету.
- 4) Підключений нормалізатор стилів [modern-normalize](#).
- 5) Код написаний з дотриманням [настанови](#).
- 6) У всіх сторінок в блоці `<head>` є метатег `viewport`.
- 7) Верстка виконана щодо трьох точок перелому: 375px, 768px і 1440px.
- 8) При написанні стилів використаний Mobile First підхід і медіа-функція `min-width`.
- 9) Стили, необхідні тільки у певному проміжку, закриті в медіа-запити `(min-width: ) and (max-width: )` або тільки `(max-width: )`.
- 10) В медіа-запитах відсутнє зайве дублювання стилів.
- 11) Виконана розмітка всіх елементів мобільного меню.
- 12) Виконано оформлення всіх елементів мобільного меню.

13) Ширина меню відповідає макету.

14) За висотою меню займає весь в'юпорт.

15) Спочатку меню приховано.

16) Якщо додати на елемент меню клас **is-open**, воно з'являється.

*Для детальнішого повторення матеріалів попередніх дисциплін, необхідних для виконання даного етапу проєкту, рекомендуємо ознайомитись із **темою 17 «Адаптивний дизайн»** дисципліни «Основи веброзробки (мови розмітки)».*

Заняття 12. Адаптив сторінки mobile-first.

Актуальність та цілі заняття

Заняття 12. «Адаптив сторінки за принципом

mobile-first. Мобільне меню. Практика»

ХІД ЗАНЯТТЯ

Сьогодні ми зосередимося на практичному застосуванні мобільного меню у проєкті. Ви закріпите навички створення розмітки, стилізації та логіки відкриття/закриття меню для мобільних пристроїв. Під час заняття будемо застосовувати mobile-first підхід, медіа-запити та класи для управління станом меню.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Мобільне меню є невід'ємною частиною сучасного адаптивного сайту, але його правильна реалізація вимагає не лише знання принципів верстки, а й практичних навичок застосування mobile-first підходу. На практичному етапі важливо навчитися створювати меню, яке не лише виглядає коректно на різних екранах, а й реагує на взаємодію користувача: відкривається, закривається, коректно обробляє кліки на фон та вибір пунктів меню.

Робота з мобільним меню на практиці допомагає студентам закріпити навички адаптивної верстки, гнучких блоків,

медіа-запитів та управління станами елементів через класи у веброботці.

ЦІЛІ ЗАНЯТТЯ

Навчитися створювати мобільне меню у проєкті макету Flora за принципом mobile-first, закріпити навички адаптивної верстки та застосування медіа-запитів, а також реалізувати логіку відкриття та закриття меню через класи та CSS-переходи.

ОСНОВНІ ЗАВДАННЯ

1. Створити базову розмітку мобільного меню згідно з [макетом](#) Flora.
2. Стилізувати меню та його елементи для мобільних екранів.
3. Забезпечити коректне відкриття/закриття меню через клас `is-open`.
4. Перевірити адаптивність та відповідність макету на різних ширинах екранів.

Підготовка до заняття

Підготовка до заняття

На даному етапі виконання свого проєкту необхідно виконати такі завдання:

- 1) При перегляді сторінки на будь-якому пристрої шириною від **320px**, не з'являється горизонтальна смуга прокручування.
- 2) Вихідний код відформатований за допомогою **Prettier**.
- 3) Всі зображення та текстовий контент взяті з макету.
- 4) Підключений нормалізатор стилів [modern-normalize](#).
- 5) Код написаний з дотриманням [настанови](#).
- 6) У всіх сторінок в блоці **<head>** є метатег **viewport**.
- 7) Верстка виконана щодо трьох точок перелому: **375px**, **768px** і **1440px**.
- 8) При написанні стилів використаний mobile-first підхід і медіа-функція **min-width**.
- 9) Стили, необхідні тільки у певному проміжку, закриті в медіа-запити (**min-width:**) та (**max-width:**) або тільки (**max-width:**).
- 10) В медіазапитах відсутнє зайве дублювання стилів.
- 11) Виконана розмітка всіх елементів мобільного меню.
- 12) Виконано оформлення всіх елементів мобільного меню.
- 13) Ширина меню відповідає макету.
- 14) За висотою меню займає весь в'юпорт.
- 15) Спочатку меню приховано.

16) Якщо додати на елемент меню клас `is-open`, воно з'являється.

Заняття 13. Ретинізація зображень.

Актуальність та цілі заняття

Заняття 13. «Ретинізація зображень.

Тренування з викладачем»

ХІД ЗАНЯТТЯ

Ми успішно перейшли до **етапу підготовки графіки для сучасних дисплеїв**. Тепер час зробити ваші зображення чіткими та привабливими на будь-якому пристрої, незалежно від щільності пікселів.

Сьогодні ми сфокусуємося на двох ключових аспектах адаптивної графіки: **респонсивності та ретинізації зображень**. Навчимося правильно масштабувати контентні та фонові зображення, підбирати графіку для екранів x1 та x2, а також використовувати медіазапити для контролю щільності пікселів. Ви закріпите навички підготовки та вставки графіки для екранів з високою щільністю пікселів (Retina/HiDPI).

Якість вашої графіки визначає не лише естетику сайту, а й зручність користування та професійний вигляд проєкту. Тож готуйтеся працювати з `img`, `background-image`, CSS-підключенням медіазапитів та властивостями `max-width` і `height: auto`, щоб створити ідеально чіткі та адаптивні зображення. Починаймо!

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Сучасні дисплеї все частіше мають високу щільність пікселів, що робить стандартні растрові зображення розмитими. Ретинізація дозволяє забезпечити чіткість графіки на всіх екранах, зберігаючи, при цьому, адаптивність сторінки. Правильне використання респонсивних та ретинізованих зображень підвищує якість UX, робить сайт професійним та відповідає сучасним стандартам веброзробки. Практична робота над ретинізацією зображень допомагає студентам закріпити навички роботи з растровою графікою, підготовки фонових і контентних зображень для різних щільностей екранів, а також застосування медіазапитів для управління графікою.

ЦІЛІ ЗАНЯТТЯ

- 1) Навчитися готувати та вставляти респонсивні зображення з підтримкою Retina.
- 2) Закріпити навички використання медіазапитів для різних щільностей пікселів.
- 3) Забезпечити коректне відображення зображень на всіх пристроях без горизонтальної прокрутки.

ОСНОВНІ ЗАВДАННЯ

1. Навчитись ретинізувати фонові та контентні зображення згідно [з макетом](#).
2. Перевірити відображення ретинізованих зображень на екранах із різною щільністю пікселів.
3. Переконатися, що всі зображення масштабуються разом із блоками і відповідають принципам адаптивності.

Підготовка до заняття

1. Основні правила адаптивної графіки

Зображення повинні масштабуватися разом із блоками. Базове правило:

```
img {  
    max-width: 100%;  
    height: auto;  
}
```

Фонові та контентні растрові зображення повинні підтримувати щільність екранів x_1 та x_2 . Для фонових зображень застосовуються медіазапити з `min-resolution` для підбору потрібної графіки.

Для ретинізації **контентних зображень** (img):

використовуй дві версії: стандартну (x1) і подвоєну роздільну здатність (x2).

підключай через `srcset`:

```

```

Для ретинізації **фонових зображень** (CSS):

пиши базове зображення для x1 і медіа-запит для x2:

```
.hero {
  background-image: url('hero@1x.jpg');
}

@media
  (-webkit-min-device-pixel-ratio: 2),
  (min-resolution: 192dpi) {
  .hero {
    background-image: url('hero@2x.jpg');
  }
}
```

2. Підготовка до практики у проєкті

Перейди за посиланням на [макет](#), з яким будемо працювати та проаналізуй детальніше його і [вимоги](#) до його розробки, а також **критерії прийняття проєкту**:

1) При перегляді сторінки на будь-якому пристрої шириною від 320px, не з'являється горизонтальна смуга прокручування.

- 2) Вихідний код відформатований за допомогою `Prettier`.
- 3) Всі зображення та текстовий контент взяті з макета.
- 4) Підключений нормалізатор стилів [modern-normalize](#).
- 5) Код написаний з дотриманням [настанови](#).
- 6) У всіх сторінок в блоці `<head>` є метатег `viewport`.
- 7) Верстка виконана щодо трьох точок перелому: `375px`, `768px` і `1440px`.
- 8) Всі фонові і контентні растрові зображення — респонсивні і підтримують екрани щільністю `x1` і `x2`.
- 9) Для респонсивних фонових зображень використані медіафункція `min-resolution`.
- 10) При написанні стилів використаний `Mobile First` підхід і медіафункція `min-width`.
- 11) Стили, необхідні тільки у певному проміжку, закриті в медіазапити `(min-width: ) and (max-width: )` або тільки `(max-width: )`.
- 12) В медіазапитах відсутнє зайве дублювання стилів.
- 13) Виконана розмітка всіх елементів мобільного меню.
- 14) Виконано оформлення всіх елементів мобільного меню.
- 15) Ширина меню відповідає макету.

16) За висотою меню займає весь в'юпорт.

17) Спочатку меню приховано.

18) Якщо додати на елемент меню клас `is-open`, воно з'являється.

3. Чеклист перевірки ретинізації

Переглянь макет та вимоги до графіки. Переконайся, що:

Всі зображення у проєкті респонсивні.

Для Retina використані версії x2 для важливих графічних елементів.

Mobile-first підхід застосований при верстці та медіазапитах.

Ретинізуючи зображення, перевір такі пункти:

Зображення не розмиті на екранах x1 і x2.

Контентні зображення масштабуються разом із блоками.

Фонові зображення відображаються коректно у всіх точках перелому (breakpoints).

Медійні запити для Retina працюють на екранах з високою щільністю пікселів.

Немає горизонтальної смуги прокручування при масштабуванні.

*Для детальнішого повторення матеріалів попередніх дисциплін, необхідних для виконання даного етапу проєкту, рекомендуємо ознайомитись із **темою 18 «Адаптивна графіка»** дисципліни «Основи веброзробки (мови розмітки)».*

Заняття 14. Ретинізація зображень.

Практика

Актуальність та цілі заняття

Заняття 14. «Ретинізація зображень. Практика»

ХІД ЗАНЯТТЯ

Доброго дня! На попередньому занятті ми розібрали, як правильно ретинізувати зображення, підбирати графіку для екранів із різною щільністю пікселів та підключати адаптивні зображення для контенту й фону.

Сьогодні час застосувати ці знання на практиці й перевірити, як ретинізація працює у твоєму проєкті **Flora**. Це заняття максимальної концентрації, присвячене аналізу твого коду, пошуку типових помилок у підключенні зображень і покращенню якості відображення графіки на x1 та x2-екранах.

Ми перевіримо: чи коректно використані `srcset` і `sizes` для контентних зображень, чи правильно реалізовані фонові зображення з урахуванням `min-resolution`, чи не з'являється спотворення пропорцій та розмиття при зміні ширини екрана. Готуй свій проєкт — сьогодні ми доводимо адаптивну графіку до рівня професійної верстки.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Сучасні дисплеї з високою щільністю пікселів (Retina, x2 та вище) здатні відображати деталі у 2–4 рази чіткіше, ніж стандартні

монітори. Якщо не підготувати графіку належним чином, зображення виглядатимуть розмитими або піксельними на смартфонах, планшетах і ноутбуках із Retina-дисплеями.

Вміння ретинізувати зображення гарантує, що сайт буде виглядати професійно на всіх пристроях, підвищує UX та відповідає сучасним стандартам веброзробки.

ЦІЛІ ЗАНЯТТЯ

Навчитися робити зображення адаптивними та чіткими для екранів x1 та x2, перевіряти якість графіки в браузері та закріпити навички використання HTML- та CSS-інструментів для керування щільністю пікселів..

ОСНОВНІ ЗАВДАННЯ

1. Навчитись масштабувати контентні та фонові зображення відповідно до макету.
2. Ретинізувати зображення у проєкті [макету](#) Flora.
3. Перевірити правильність відображення графіки на екранах x1/x2 та різних точках перелому.

Підготовка до заняття

- 1. Підготовка контентних зображень для Retina-екранів**

Перед початком практики необхідно пригадати, як підключаються контентні зображення з підтримкою різної щільності пікселів. Для цього використовується атрибут `srcset`, який дозволяє браузеру автоматично обрати відповідну версію зображення для екранів x1 або x2.

Базовий приклад підключення:

```

```

Важливо, щоб фізичний розмір файлу `@2x` був у два рази більшим за версію `@1x`, але візуально на сторінці вони відображались в одному й тому самому розмірі.

Для коректного масштабування зображень у верстці обов'язково використовується правило:

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

2. Ретинізація фонових зображень у CSS

Фонові зображення не підтримують `srcset`, тому для них використовується підміна зображень через медіазапити за щільністю пікселів. Базовий підхід:

```
.section {  
    background-image: url("../images/bg@1x.jpg");  
    background-size: cover;  
    background-position: center;  
}  
  
@media (min-resolution: 2dppx) {  
    .section {  
        background-image: url("../images/bg@2x.jpg");  
    }  
}
```

Таким чином для екранів з щільністю $\times 2$ автоматично буде підвантажуватись більш якісна версія фонового зображення.

3. Поєднання ретинізації та mobile-first підходу

Під час підготовки до заняття важливо пригадати, що ретинізація має обов'язково поєднуватись із mobile-first підходом.

Спочатку задаються стилі та зображення для найменших екранів, а далі через медіазапити `min-width` і `min-resolution` розширюється логіка підключення графіки. Приклад поєднання:

```
.hero {  
    background-image:  
url("../images/hero-mobile@1x.jpg");
```



```

    }

    @media (min-resolution: 2dppx) {
        .hero {
            background-image:
            url("../images/hero-mobile@2x.jpg");
        }
    }

    @media (min-width: 768px) {
        .hero {
            background-image:
            url("../images/hero-tablet@1x.jpg");
        }
    }

    @media (min-width: 768px) and (min-resolution: 2dppx)
    {
        .hero {
            background-image:
            url("../images/hero-tablet@2x.jpg");
        }
    }

```

Такий підхід дозволяє одночасно враховувати і ширину екрану, і його щільність пікселів.

4. Перевірка ретинізації

Перед початком практичної роботи необхідно пригадати, як перевіряти коректність підвантаження зображень:

відкрити DevTools → вкладка *Network* → *Img*;

змінити Device Pixel Ratio у емуляторі пристроїв;
перевірити, чи підвантажується файл [@1x](#) або [@2x](#)
відповідно до щільності екрану;
переконатися, що візуальний розмір зображення не
змінюється.

На даному етапі виконання свого проєкту необхідно **виконати такі завдання:**

- 1) При перегляді сторінки на будь-якому пристрої шириною від 320px, не з'являється горизонтальна смуга прокручування.
- 2) Вихідний код відформатований за допомогою [Prettier](#).
- 3) Всі зображення та текстовий контент взяті з макета.
- 4) Підключений нормалізатор стилів [modern-normalize](#).
- 5) Код написаний з дотриманням [настанови](#).
- 6) У всіх сторінок в блоці `<head>` є метатег `viewport`.
- 7) Верстка виконана щодо трьох точок перелому: 375px, 768px і 1440px.
- 8) Всі фонові й контентні растрові зображення — респонсивні і підтримують екрани щільністю `x1` і `x2`.
- 9) Для респонсивних фонових зображень використана медіафункція [min-resolution](#).
- 10) При написанні стилів використаний `mobile-first` підхід і медіафункція [min-width](#).

- 11) Стили, необхідні тільки у певному проміжку, закриті в медіа-запити (`min-width:`) та (`max-width:`) або тільки (`max-width:`).
- 12) В медіазапитах відсутнє зайве дублювання стилів.
- 13) Виконана розмітка всіх елементів мобільного меню.
- 14) Виконано оформлення всіх елементів мобільного меню.
- 15) Ширина меню відповідає макету.
- 16) За висотою меню займає весь в'юпорт.
- 17) Спочатку меню приховано.
- 18) Якщо додати на елемент меню клас `is-open`, воно з'являється.

Заняття 15. Модальні вікна та робота з формами.

Актуальність та цілі заняття

Заняття 15. «Модальні вікна та робота з формами. Тренування з викладачем»

ХІД ЗАНЯТТЯ

Доброго дня! Ми пройшли теорію: навчилися правильно створювати модальні вікна, працювати з формами, забезпечувати їхню семантичність і доступність. Сьогодні час застосувати ці знання на практиці й побачити реальний ефект у проєкті.

Це буде заняття максимальної концентрації, присвячене перевірці вашого коду, виявленню помилок у верстці форм та модальних вікон і закріпленню навичок адаптивної верстки.

Ми перевіримо: чи коректно створений бекдроп модального вікна, чи правильно спозиційоване вікно по центру, чи працює відкриття/закриття через клас `is-open`, а також чи всі інпути, кнопки і чекбокси мають правильні атрибути, пов'язані `<label>` та стилі фокусу і ховера. Готуй свій проєкт — сьогодні ми закріплюємо наші навички роботи з інтерфейсними елементами, які підвищують UX і якість сайту.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Модальні вікна і форми — критично важливі елементи сучасного сайту: вони дозволяють організувати інтерактивну взаємодію з користувачем, збирати дані й підвищувати конверсію. Навички правильної їхньої верстки та стилізації під макет гарантують, що користувач отримає зручний і передбачуваний інтерфейс. Уміння робити семантичні форми та адаптивні модальні вікна також забезпечує доступність сайту та полегшує подальшу підтримку коду.

ЦІЛІ ЗАНЯТТЯ

1. Навчитися створювати модальні вікна відповідно до макету.
2. Закріпити навички роботи з формами у футері та модальних вікнах.
3. Навчитися контролювати відкриття/закриття модального вікна через класи та CSS-переходи.
4. Забезпечити адаптивність і коректне відображення форм та модальних вікон на різних пристроях.

ОСНОВНІ ЗАВДАННЯ

1. Навчитись згідно [макету](#) створювати бекдроп і модальне вікно.
2. Реалізувати адаптивні форми у футері та модальних вікнах.
3. Забезпечити правильну поведінку елементів при відкритті/закритті модального вікна.

4. Стилїзувати елементи форм з урахуванням фокусу, ховера та SVG-їконок.

5. Перевїрити семантику і доступність усіх елементів.

Підготовка до заняття

1. Розмїтка бекдропа та модального вїкна

На першому етапі роботи потрібно створити базову структуру модального вїкна.

Важливо зрозумїти, що бекдроп — це темний напівпрозорий фон, який покриває весь екран і слугує фоном для модального вїкна. Модальне вїкно розмїщується всерединї бекдропа і спозицїоноване по центру екрану.

Для цього потрібно додати контейнер бекдропа, який має властивостї `position: fixed; top: 0; left: 0; width: 100%; height: 100%;`, та розмїстити всерединї нього блок модального вїкна з адаптивними шириною і висотою. Кнопка закриття розмїщується у верхньому правому кутї, а її натискання або натискання на фон бекдропа має закривати модалку. Початковий стан елементів — прихований, що реалїзується через клас `is-open`.

2. Стилїзацїя модального вїкна та бекдропу

Другий етап полягає у вїзуальному оформленнї. Бекдроп повинен мати напівпрозорий фон (наприклад, `rgba(0, 0, 0, 0.5)`) та

фіксоване позиціонування, щоб залишатися на місці під час прокрутки сторінки. Модальне вікно повинно відповідати ширині макету і зберігати відступи на всіх екранах.

Особливу увагу слід приділити кнопці закриття — вона повинна бути видимою, зручною для натискання та доступною через клавіатуру. Всі елементи модалки повинні коректно масштабуватися на різних пристроях і відповідати кольоровій палітрі та шрифтам макету.

3. Логіка відкриття та закриття

Для того, щоб модальне вікно працювало динамічно, необхідно навчитися керувати його станом через CSS-клас `is-open`. Додавання цього класу відкриває модалку і показує бекдроп.

Важливо реалізувати закриття модального вікна при натисканні на кнопку закриття або на бекдроп. При цьому, слід враховувати, щоб сторінка не прокручувалася горизонтально, не з'являлися додаткові смуги прокручування й користувацький досвід залишався зручним.

4. Створення та стилізація форм

Форми підписки та заявки повинні бути семантичними і доступними. Кожен інпут має мати атрибут `name` з описовим

значенням та пов'язаний `<label>`. Кнопки відправлення форм мають мати `type="submit"`.

Чекбокси, такі як прийняття ліцензійної угоди, необхідно стилізувати вручну, використовуючи SVG, приховавши стандартний input. Для всіх елементів форми повинні бути додані hover/focus-ефекти з плавними CSS-переходами. Це забезпечує інтуїтивність взаємодії та відповідність макету.

5. Адаптивність та перевірка

Останній етап — перевірка адаптивності. Модальні вікна та форми повинні коректно відображатися на мобільних пристроях (320–375px), планшетах (768px) та десктопах (1440px).

Важливо впевнитися, що модалка не виходить за межі viewport, всі елементи масштабуються пропорційно, а клавіатурна навігація та атрибути aria забезпечують доступність для всіх користувачів.

Перейди за посиланням на [макет](#), з яким будемо працювати та проаналізуй детальніше його і [вимоги](#) до його розробки, а також **критерії прийняття проєкту**:

- 1) Всі стилі написані в одному файлі `styles.css`, який знаходиться в папці `css`.
- 2) Вихідний код відформатований за допомогою `Prettier`.

- 3) Всі зображення та текстовий контент взяті з макета.
- 4) Підключений нормалізатор стилів [modern-normalize](#).
- 5) Код написаний з дотриманням [настанови](#).
- 6) Виконана розмітка і оформлення «бекдропа» (темного напівпрозорого фону) модального вікна.
- 7) «Бекдроп» заповнює 100% висоти і ширини в'юпорту браузера і фіксований в ньому.
- 8) Виконана розмітка і оформлення модального вікна.
- 9) Модальне вікно вертикально і горизонтально спозиційоване посередині бекдропа.
- 10) Виконана розмітка і оформлення кнопки закриття модального вікна у верхньому правому куті.
- 11) При початковому стані модальне вікно і бекдроп приховані.
- 12) При додаванні на елемент бекдроп клас `is-open` — з'являється бекдроп і модальне вікно.
- 13) Виконана HTML-розмітка всіх елементів макета.
- 14) Теги використані відповідно до їх семантичного змісту.
- 15) Виконана розмітка форми підписки на розсилання і всіх її елементів у футері.

- 16) Виконана розмітка форми заявки та всіх її елементів в модальному вікні.
- 17) У всіх інпутів у формах заданий атрибут `name`.
- 18) Всі значення атрибута `name` — описові та точно характеризують, для чого це поле форми.
- 19) У всіх інпутів є пов'язаний елемент `<label>`.
- 20) Інпутам заданий атрибут `placeholder`, якщо для нього в макеті є текст-підказка.
- 21) Кнопкам «відправлення» форм заданий атрибут `type="submit"`.
- 22) Всі нові іконки з форм додані в SVG-спрайт `icons.svg`.
- 23) Виконано оформлення елементів форми підписки на розсилання у футері.
- 24) Виконано оформлення елементів форми заявки у модальному вікні.
- 25) При отриманні інпутом фокусу, його рамка та іконка змінюють колір (як показано на макеті).
- 26) Оригінальний чекбокс про прийняття ліцензійної угоди у формі заявки прихований.

27) Оформлення «чекбокса» про прийняття ліцензійної угоди зроблено вручну, за допомогою векторного зображення галочки з SVG-спрайту.

28) Для всіх ефектів хOVERа і фокуса (колір, фон, тінь) зроблені переходи. Час — 250ms, функція розподілу часу — cubic-bezier(0.4, 0, 0.2, 1).

Для перевірки коректності виконаного завдання варто:

Перевірити модальне вікно та форми на мобільних пристроях (320–375px), планшетах (768px) та десктопах (1440px).

Переконавшись, що модалка не виходить за межі viewport та адаптивно реагує на розмір екрану.

Перевірити доступність такого функціоналу: клавіатурна навігація, focus та aria-атрибути для користувачів із обмеженими можливостями.

*Для детальнішого повторення матеріалів попередніх дисциплін, необхідних для виконання даного етапу проєкту, рекомендуємо ознайомитись із **темою 15 «Форми»** дисципліни «Основи веброзробки (мови розмітки)».*

Заняття 16. Модальні вікна та робота з формами.

Актуальність та цілі заняття

Заняття 16. «Модальні вікна та робота з формами. Практика»

ХІД ЗАНЯТТЯ

Доброго дня! Ми пройшли теорію про семантичні теги для позначення форм та принципи UX для інтерфейсів.

Сьогодні ми застосуємо ці знання на практиці і створимо функціональні модальні вікна та форми для сайту Flora. Заняття присвячене максимальній концентрації на розмітці, стилізації та поведінці елементів, які формують користувацький досвід.

Ми навчимося позиціонувати модальні вікна по центру екрану, керувати їх станом через CSS-класи, оформляти форми для різних пристроїв та перевіряти адаптивність на різних ширинах екранів.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Модальні вікна та форми є ключовими елементами взаємодії користувача із сайтом. Вони дозволяють реалізувати швидкий доступ до додаткових дій, таких як підписка на розсилку, замовлення або контакт із компанією, без необхідності перенаправляти користувача на іншу сторінку.

Правильна реалізація модальних вікон підвищує зручність користування сайтом і робить інтерфейс більш інтуїтивним. При цьому важливо не лише знати CSS та HTML, але й розуміти принципи адаптивної верстки та UX. Навички, які ви здобудете сьогодні, дозволять створювати професійний і функціональний інтерфейс, який однаково добре працює на мобільних пристроях, планшетах і десктопах.

ЦІЛІ ЗАНЯТТЯ

Навчитися створювати адаптивні й зручні модальні вікна та форми для вебпроєкту.

ОСНОВНІ ЗАВДАННЯ

1. Навчитися правильно розміщувати модальне вікно на екрані, керувати його станом через CSS-класи у проєкті [макету](#) Flora.
2. Оформити форми підписки та заявки відповідно до макету та забезпечити їхню доступність. В результаті ви отримаєте повний набір практичних навичок, необхідних для інтерактивних вебінтерфейсів.

Підготовка до заняття

1. Для розмітки бекдропу та модального вікна потрібно:

Створити контейнер бекдропу, який покриває весь viewport.
Додати модальне вікно всередину бекдропу та спозиціонувати його по центру екрану.
Додати кнопку закриття у верхньому правому куті модального вікна.
Початковий стан бекдропу та модалки має бути прихованим.

2. Щоб стилізувати модальне вікно та бекдроп, необхідно:

Перевірити, щоб бекдроп мав напівпрозорий фон та фіксоване позиціонування.
Модальне вікно повинно відповідати ширині макету на різних пристроях, з адаптивними відступами.
Кнопка закриття має бути інтерактивною і доступною для клавіатурної навігації.

3. Далі необхідно **реалізувати логіку відкриття та закриття:**

Для показу модального вікна додавати клас `is-open` на бекдроп і модалку.
При натисканні на кнопку закриття або на фон бекдропу клас `is-open` потрібно прибрати.
Врахувати, щоб при відкритті модалки сторінка не прокручувалася горизонтально і не з'являлися непотрібні лінії прокручування (скроли).

4. Для створення та стилізації форм потрібно:

Додати семантичну HTML-розмітку для форми підписки та форми заявки.

Всі інпути повинні мати атрибут `name` та пов'язані `<label>`.
Кнопки відправлення повинні мати `type="submit"`.
Оформлення чекбоксу ліцензійної угоди зробити вручну,
через SVG, приховавши стандартний input.
Додати hover/focus-ефекти для всіх елементів форм з
плавними CSS-переходами.

На даному етапі виконання свого проєкту необхідно **виконати завдання за такими критеріями:**

- 1) Всі стилі написані в одному файлі `styles.css`, який знаходиться в папці `css`.
- 2) Вихідний код відформатований за допомогою `Prettier`.
- 3) Всі зображення та текстовий контент взяті з макета.
- 4) Підключений нормалізатор стилів [modern-normalize](#).
- 5) Код написаний з дотриманням [настанови](#).
- 6) Виконана розмітка і оформлення бекдропа (темного напівпрозорого фону) модального вікна.
- 7) Бекдроп заповнює 100% висоти і ширини в'юпорту браузера і фіксований в ньому.
- 8) Виконана розмітка і оформлення модального вікна.
- 9) Модальне вікно вертикально і горизонтально спозиційоване посередині бекдропа.

- 10) Виконана розмітка та оформлення кнопки закриття модального вікна у верхньому правому куті.
- 11) При початковому стані модальне вікно і бекдроп приховані.
- 12) Якщо додати на елемент бекдроп клас `is-open` — з'являється бекдроп і модальне вікно.
- 13) Виконана HTML-розмітка всіх елементів макета.
- 14) Теги використані відповідно до їх семантичного змісту.
- 15) Виконана розмітка форми підписки на розсилання і всіх її елементів у футері.
- 16) Виконана розмітка форми заявки та всіх її елементів в модальному вікні.
- 17) У всіх інпутів у формах заданий атрибут `name`.
- 18) Значення атрибута `name` описові, що точно характеризують, для чого це поле форми.
- 19) У всіх інпутів є пов'язаний елемент `<label>`.
- 20) Інпутам заданий атрибут `placeholder`, якщо для нього в макеті є текст-підказка.
- 21) На всіх формах кнопкам «Відправити» заданий атрибут `type="submit"`.
- 22) Всі нові іконки з форм додані в SVG-спрайт `icons.svg`.

23) Виконано оформлення елементів форми підписки на розсилання у футері.

24) Виконано оформлення елементів форми заявки у модальному вікні.

25) При отриманні інпуту фокусу, його рамка та іконка змінюють колір (як показано на макеті).

26) Оригінальний чекбокс про прийняття ліцензійної угоди у формі заявки прихований.

27) Оформлення «чекбокса» про прийняття ліцензійної угоди зроблено вручну, за допомогою векторного зображення галочки з SVG-спрайту.

28) Для всіх ефектів ховера і фокуса (колір, фон, тінь) зроблені переходи. час — `250ms`, функція розподілу часу — `cubic-bezier(0.4, 0, 0.2, 1)`.

Заняття 17. Робота із запитами.

Актуальність та цілі заняття

Заняття 17. «Робота із запитами. Динамічна розмітка. Тренування з викладачем»

ХІД ЗАНЯТТЯ

Доброго дня! На попередньому занятті ми завершили роботу з формами та модальними вікнами і навчилися правильно будувати інтерфейс для взаємодії користувача зі сторінкою.

Сьогодні ми переходимо до наступного важливого етапу — роботи з динамічними списками та даними, отриманими з сервера. Ми навчимося виконувати HTTP-запити за допомогою бібліотеки `axios`, працювати з асинхронним кодом через `async/await` та на основі отриманих даних формувати розмітку списків безпосередньо в JavaScript.

Під час заняття ми зосередимося на створенні динамічних елементів інтерфейсу (карток, списків, блоків), їх коректному додаванні до DOM і відповідності структурі та дизайну макету Flora. Окрему увагу приділимо перевірці коректності отриманих даних, обробці помилок запитів і контролю порядку відмальовування елементів.

Мета сьогоднішньої практики — навчитися перетворювати статичну HTML-сторінку на сторінку з динамічним наповненням

списків, що працює з реальними даними та відповідає сучасним підходам до побудови клієнтської частини вебзастосунків. Готуйте свій проєкт — починаймо!

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

У сучасному вебі практично всі сайти працюють із динамічним контентом: новини, товари, користувачі, коментарі — усе це отримується через запити до серверу. Вміння робити запити та формувати розмітку на основі отриманих даних дозволяє створювати швидкі, інтерактивні та адаптивні сторінки. Без цього навіку сайт буде статичним, а взаємодія користувача обмеженою.

Практичні навички роботи з динамічними списками та асинхронними даними — обов'язкові для сучасного фронтенд-розробника.

ЦІЛІ ЗАНЯТТЯ

1. Навчитися виконувати HTTP-запити та отримувати дані у форматі JSON.
2. Закріпити навички асинхронної роботи з JavaScript через `async/await`.
3. На основі отриманих даних формувати динамічну розмітку сторінки.

4. Навчитися перевіряти та обробляти помилки запитів.

ОСНОВНІ ЗАВДАННЯ

1. Навчитись динамічно формувати списки елементів згідно [макету](#) Flora, отримуючи дані через HTTP-запити.
2. Відпрацювати правильне використання `async/await` для асинхронних операцій.
3. Перевірити коректність вставки елементів у DOM та уникнути дублювання.
4. Навчитися обробляти помилки при запитах і відображати користувачеві повідомлення у разі недоступності даних.

Підготовка до заняття

1. Використання бібліотеки `axios`

Для HTTP-запитів ми будемо використовувати бібліотеку `axios`. Переконайся, що вона підключена у проєкті або встановлена через `npm/yarn`.

Приклад підключення через CDN:

```
<script  
src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"  
></script>
```

2. Використання `async/await`

Для асинхронних операцій будемо застосовувати `async/await`:

```
async function getData() {
  try {
    const response = await
axios.get('https://api.example.com/items');
    renderItem(response.data);
  } catch (error) {
    console.error('Помилка запиту:', error);
  }
}
```

Це дозволяє легко читати код і контролювати виконання запитів.

3. Формування динамічної розмітки

Оскільки бекенд-частина ще не розгорнута та відсутні вихідні дані для створення динамічної розмітки, скористаємось пакетом [json-server](#) для розгортання локального серверу. Створити файл `db.json` з даними, взятими з цього [ресурсу](#).

Отримані дані потрібно перетворити у HTML за допомогою функцій, циклів і шаблонних рядків:

```
function renderItem(items) {
  const list = document.querySelector('.item-list');
  list.innerHTML = ''; // Очищаємо перед вставкою нових елементів
  items.forEach(item => {
    list.insertAdjacentHTML('beforeend', `
    <li class="item">
      <h3>${item.title}</h3>
      <p>${item.description}</p>
    `);
  });
}
```



```
        </li>  
    `);  
    });  
}
```

4. Обробка помилок та UX

Під час виконання HTTP-запитів дані можуть не завантажитися (мережеві збої, помилка сервера, порожня відповідь). Твоє завдання — зробити так, щоб інтерфейс коректно реагував на ці стани й не «ламавав» динамічну розмітку. Що потрібно реалізувати:

1) Стан «Завантаження»

перед стартом запиту показуй індикатор (spinner/loader);
після завершення запиту (успіх/помилка) — прибитай індикатор.

2) Стан «Помилка»

якщо запит завершився помилкою — виведи зрозуміле повідомлення для користувача (не технічний текст);
UI має залишатися керованим: користувач розуміє, що сталося, і що робити далі (наприклад, “Спробуйте ще раз”).

3) Стан «Успішне завантаження» без дублювання

перед повторним рендером очищай контейнер або оновлюй розмітку так, щоб елементи не дублювалися при повторному запиті чи перезавантаженні даних.

Очікуваний результат: під час кожного запиту користувач бачить, що відбувається (loading), отримує адекватний фідбек у разі збою (error) і не спостерігає «наростання» однакових елементів у списку (no duplicates).

Перейди за посиланням на [макет](#), з яким будемо працювати та проаналізуй детальніше його й [вимоги](#) до його розробки, а також **критерії прийняття проєкту**:

- 1) Для HTTP-запитів використовуй бібліотеку [axios](#).
- 2) Використовуй синтаксис `async/await`.

*Для детальнішого повторення матеріалів попередніх дисциплін, необхідних для виконання даного етапу проєкту, рекомендуємо ознайомитись із темами дисципліни «Основи веброзробки (JavaScript)», зокрема **темою 13 «Модель DOM»**.*

Заняття 18. Робота із запитами.

Актуальність та цілі заняття

Заняття 18. «Робота із запитами. Динамічна розмітка. Практика»

ХІД ЗАНЯТТЯ

Доброго дня!

Ми пройшли базову теорію роботи з DOM та асинхронними запитами у JavaScript, а сьогодні час застосувати ці знання на практиці. Сьогоднішнє заняття максимально зосереджене на роботі з динамічною розміткою: ви навчитеся отримувати дані через HTTP-запити, перетворювати їх у HTML і вставляти у сторінку, а також організовувати повторювані елементи списків за допомогою шаблонів та циклів.

Це буде заняття, де важлива точність: від правильності запиту до формування елементів залежить, чи зможе сайт правильно відобразити контент у будь-який момент часу. Готуй свій проєкт — сьогодні ми закріплюємо навички динамічної роботи з даними на сторінці.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Сучасні вебсайти все рідше використовують статичну розмітку й все частіше наповнюються даними з серверів у режимі реального часу. Саме тому вміння виконувати HTTP-запити,

обробляти відповіді та формувати динамічні списки без перезавантаження сторінки є базовою компетенцією фронтенд-розробника.

Практична робота з динамічною розміткою дозволяє закріпити навички взаємодії з DOM, асинхронного програмування та коректної побудови інтерфейсу відповідно до макету, що є необхідним для створення інтерактивних і масштабованих вебпроектів.

ЦІЛІ ЗАНЯТТЯ

Навчитися отримувати дані з сервера за допомогою HTTP-запитів, створювати на їх основі динамічні списки елементів сторінки та коректно інтегрувати їх у структуру проекту згідно з макетом.

ОСНОВНІ ЗАВДАННЯ

1. Навчитися виконувати асинхронні запити до API за допомогою бібліотеки `axios` та обробляти отримані дані.
2. Реалізувати динамічне формування списків елементів у проекті [макету](#) Flora.

Підготовка до заняття

- 1. Підготовка проекту до роботи з HTTP-запитами**

Перед початком практичного заняття необхідно *підготувати проєкт до роботи з асинхронними запитами до сервера*. У структурі проєкту має бути підключена бібліотека `axios`, яка буде використовуватися для виконання HTTP-запитів.

Важливо переконатися, що *JavaScript-файл коректно підключений до HTML-сторінки* та не містить синтаксичних помилок, які можуть завадити виконанню запитів або обробці отриманих даних.

Також необхідно перевірити, що *середовище розробки дозволяє працювати з модулями* або зовнішніми бібліотеками відповідно до налаштувань проєкту.

2. Підготовка до роботи з асинхронною логікою та обробкою даних

Для успішного виконання практичного завдання вам потрібно пригадати принципи асинхронного програмування в JavaScript та особливості використання синтаксису `async/await`. Під час заняття всі запити до сервера мають виконуватися у вигляді асинхронних функцій із коректною обробкою результату запиту та можливих помилок.

Оскільки бекенд-частина ще не розгорнута та відсутні вихідні дані для створення динамічної розмітки, скористайтесь пакетом

[json-server](#) для розгортання локального серверу. Створити файл `db.json` з даними, взятими з цього [ресурсу](#).

Отримані дані перетворити у HTML за допомогою функцій і шаблонних рядків, наприклад:

```
function renderItem(items) {
  const list = document.querySelector('.item-list');
  list.innerHTML = ''; // Очищаємо перед вставкою нових
елементів
  items.forEach(item => {
    list.insertAdjacentHTML('beforeend', `
      <li class="item">
        <h3>${item.title}</h3>
        <p>${item.description}</p>
      </li>
    `);
  });
}
```

Окрему увагу слід приділити розумінню структури відповіді сервера та способам отримання необхідних полів з об'єкта відповіді для подальшого формування розмітки.

3. Підготовка до формування динамічних списків елементів сторінки

Перед практикою необхідно повторити принципи роботи з DOM-деревом та способи створення елементів інтерфейсу за допомогою JavaScript. Треба бути готовим до побудови

повторюваних елементів списку на основі масивів даних, отриманих із сервера.

Особливу увагу слід звернути на використання циклів і шаблонних рядків для генерації HTML-розмітки, а також на коректне вставлення згенерованого контенту у відповідні контейнери сторінки. У межах заняття статична HTML-розмітка для елементів списку не використовується — всі такі елементи формуються динамічно.

4. Перевірка відповідності реалізації вимогам проекту

Перед початком виконання практичного завдання необхідно ознайомитися з макетом сторінки Flora та визначити блоки, у яких контент має формуватися динамічно.

Під час підготовки слід перевірити, що структура HTML дозволяє коректно вставляти списки елементів без порушення семантики та стилів сторінки. Динамічно створені елементи повинні відповідати структурі та оформленню, передбаченим у макеті, а логіка отримання та відображення даних має бути інтегрована в існуючу архітектуру проекту без дублювання коду.

Перевір, що у проекті підключена бібліотека `axios` та налаштоване середовище для роботи з JavaScript-файлами.

На даному етапі виконання свого проєкту необхідно **виконати**
такі завдання:

Для виконання HTTP-запитів у проєкті використовується бібліотека axios.

Асинхронна логіка реалізована із застосуванням синтаксису async/await.

Динамічні списки елементів створюються на основі отриманих даних та вставляються у DOM без використання статичної HTML-розмітки для цих елементів.

Заняття 19. Пагінація. Фільтрація

Актуальність та цілі заняття

Заняття 19. «Пагінація. Фільтрація. Тренування

з викладачем»

ХІД ЗАНЯТТЯ

Ми вже навчилися отримувати дані з сервера та формувати динамічні списки елементів на сторінці. Тепер настав час зробити цю взаємодію з даними по-справжньому зручною для користувача.

Сьогодні ми зосередимося на реалізації пагінації та керуванні результатами запитів, щоб сторінка не перевантажувалася великою кількістю елементів, а контент підвантажувався поступово.

Під час заняття ви навчитеся працювати з параметрами сторінок у запитах, керувати станом поточного пошуку, обробляти кінець колекції та організовувати плавну взаємодію інтерфейсу з користувачем. Також ви закріпите навички роботи з бібліотеками для відображення контенту в модальному режимі.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Сучасні вебзастосунки працюють із великими обсягами даних, тому відображення всієї інформації одразу є неефективним як з точки зору продуктивності, так і з точки зору користувацького

досвіду. Пагінація та фільтрація дозволяють організувати поетапне завантаження контенту, зменшити навантаження на інтерфейс і забезпечити зручну навігацію результатами пошуку.

Вміння реалізовувати механізми посторінкового завантаження, керувати станом запитів та коректно обробляти кінець вибірки є обов'язковими навичками фронтенд-розробника під час створення вебпроектів.

ЦІЛІ ЗАНЯТТЯ

Сформувати практичні навички реалізації пагінації та керування результатами пошуку, навчитися організовувати підвантаження даних за запитом користувача та коректно відображати динамічні списки відповідно до макету.

ОСНОВНІ ЗАВДАННЯ

1. Навчитись реалізовувати пагінацію та підвантаження даних за допомогою кнопки Load more згідно [з макетом](#).
2. Реалізувати механізм посторінкового завантаження та фільтрації результатів у проєкті макету Flora.
3. Навчитись керувати станом пошуку та обробляти завершення колекції результатів

Підготовка до заняття

Підготовка до заняття

Перейди за посиланням на [макет](#), з яким будемо працювати, та проаналізуй його структуру, блоки динамічного контенту й [вимоги](#) до реалізації інтерфейсу.

Під час підготовки зверни увагу на такі **вимоги до реалізації**:

1) Додай в HTML-документі після галереї розмітку кнопки з текстом *Load more*, за кліком на яку необхідно виконувати запит за наступною групою зображень і додавати розмітку до вже існуючих елементів галереї. Для цього при сабміті форми тобі необхідно зберігати те, що ввів користувач у глобальну змінну.

Поки в галерії нема зображень, кнопка повинна бути прихована.

Після того як у галереї з'являються зображення, кнопка з'являється в інтерфейсі під галереєю.

При повторному сабміті форми кнопка спочатку ховається, а після отримання результатів запиту знову відображається за потреби.

Перенеси індикатор завантаження під кнопку завантаження додаткових зображень.

2) Зроби так, щоб у кожній відповіді при пошуку зображень приходило 15 об'єктів (за замовчуванням 20).

Початкове значення параметра `page` повинно бути 1.

З кожним наступним запитом його необхідно збільшити на 1.

У разі пошуку за новим ключовим словом значення `page` потрібно повернути до початкового, оскільки буде пагінація для нової колекції зображень.

3) У відповіді бекенд повертає загальну кількість статей, які відповідають критерію пошуку. Якщо користувач дійшов до кінця колекції, ховай кнопку `Load more` і виводь повідомлення з наступним текстом.

We're sorry, but you've reached the end of search results.

Зверни увагу, що кінець колекції може бути як на 1-й сторінці, так й на подальших.

4) Зроби плавне прокручування сторінки після запиту і відтворення кожної наступної групи зображень. Для цього отримай у коді висоту однієї карточки галереї, використовуючи функцію `getBoundingClientRect`. Після цього використовуй метод `window.scrollTo` для прокрутки сторінки на дві висоти карточки галереї.

Критерії прийняття завдання:

- 1) Усі асинхронні запити відрефакторені і реалізовані за допомогою синтаксиса `async/await`
- 2) За один запит у відповідь приходить 15 елементів
- 3) Нові елементи додаються в DOM за одну операцію

- 4) На сторінці під галереєю міститься кнопка «Load more», при кліку на яку відправляється запит на наступну сторінку.
- 5) Після додавання нових елементів до списку зображень на екземплярі SimpleLightbox викликається метод `refresh()`
- 6) Коли користувач отримує результати за максимально можливою сторінкою для конкретного пошукового слова, тобто вже немає чого підвантажувати, кнопка «Load more» зникає і з'являється відповідне повідомлення.
- 7) При новому сабміті форми сторінка повертається до 1-ї, а попередні результати не відображаються.
- 8) При кліку на маленьке зображення в галереї відкривається його збільшена версія у модальному вікні з використанням бібліотеки SimpleLightbox.

*Для детальнішого повторення матеріалів попередніх дисциплін, необхідних для виконання даного етапу проєкту, рекомендуємо ознайомитись із **темами 21-24 дисципліни «Основи веброзробки (Java Script)».***

Заняття 20. Пагінація. Фільтрація.

Актуальність та цілі заняття

Заняття 20. «Пагінація. Фільтрація. Практика»

ХІД ЗАНЯТТЯ

Доброго дня!

На попередньому занятті ми навчилися реалізовувати пагінацію та базову фільтрацію під час отримання даних із сервера. Сьогодні ж ми переходимо до повноцінної практики та зосереджуємося на поєднанні цих двох механізмів у реальному проєкті Flora.

Протягом заняття ви будете працювати зі списками даних, які формуються динамічно на основі HTTP-запитів, і навчитесь синхронізувати зміну сторінок із параметрами фільтрації. Ми розберемо, як коректно скидати номер сторінки при зміні фільтрів, як оновлювати інтерфейс без перезавантаження сторінки та як уникати дублювання елементів у списках.

Особливу увагу приділимо організації стану застосунку: зберіганню поточних параметрів пошуку, активних фільтрів і номера сторінки. Ви побачите, як правильна структура коду спрощує підтримку логіки запитів та роботу з динамічною розміткою.

Це заняття спрямоване на відпрацювання реальних сценаріїв користувацької взаємодії зі списками — фільтрації результатів, переходів між сторінками та повторного завантаження даних. Готуй свій проєкт — сьогодні ми доводимо механізми пагінації та фільтрації до стабільної, зрозумілої та професійної реалізації.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

У сучасних вебпроєктах робота з великими масивами даних неможлива без зручної навігації між результатами та інструментів їхнього відбору. Саме тому механізми пагінації та фільтрації є обов'язковими складовими інтерфейсу користувача. На прикладі проєкту за макетом Flora ми закріпимо навички практичної реалізації динамічного списку елементів, адаптивної верстки та взаємодії інтерфейсу з даними, що надходять із запитів. Особлива увага приділяється коректній роботі елементів керування у різних розмірах екранів і відповідності реалізації вимогам макету.

ЦІЛІ ЗАНЯТТЯ

Закріплення практичних навичок реалізації пагінації та фільтрації в інтерфейсі вебзастосунку, а також відпрацювання адаптивної верстки сторінок проєкту Flora з урахуванням вимог макету та сучасних підходів до побудови UI.

ОСНОВНІ ЗАВДАННЯ

1. Відпрацювати практичну реалізацію механізмів пагінації та фільтрації даних у клієнтській частині застосунку з подальшим коректним відображенням результатів у вигляді динамічного списку.
2. Реалізувати адаптивну верстку сторінок у проєкті на [макеті](#) Flora, забезпечивши коректну поведінку інтерфейсу на різних типах пристроїв і розмірах екранів.

Підготовка до заняття

Підготовка до заняття

*На даному етапі виконання власного проєкту необхідно заздалегідь підготувати макет та кодову базу для практичної роботи на занятті. Перед початком роботи рекомендується детально **проаналізувати структуру сторінок** макету Flora, розташування елементів списків і блоків фільтрації, а також вимоги до їхньої поведінки в адаптивному режимі.*

У процесі підготовки до заняття потрібно виконати такі дії.

1. Аналіз макету та логіки відображення списків. Необхідно визначити, які саме елементи сторінки формують динамічний список, що підлягає пагінації, та які параметри впливають на фільтрацію результатів. Слід звернути увагу на взаємне розташування елементів керування, контейнера списку та

службових повідомлень інтерфейсу, а також продумати, як ці блоки повинні змінювати своє розміщення на різних ширинах екрана.

2. Аналіз вимог до реалізації пагінації у проєкті Flora. Перед початком практичного заняття необхідно ознайомитися з макетом проєкту Flora та визначити, які саме списки даних повинні завантажуватися частинами. Слід звернути увагу на наявність елементів керування завантаженням даних (кнопка переходу між сторінками або кнопка завантаження наступної порції результатів), а також на логіку їх відображення в інтерфейсі.

Окремо потрібно проаналізувати, у який момент інтерфейс має оновлюватися після отримання відповіді від сервера та як поводить ся список елементів при переході між сторінками результатів.

3. Підготовка логіки фільтрації даних на стороні клієнта. Необхідно повторити принцип роботи фільтрації через параметри HTTP-запиту та зрозуміти, яким чином значення з елементів керування (select, input, checkbox або форма) впливають на сформований запит до сервера. Слід підготувати логіку формування об'єкта параметрів фільтрації та визначити, які саме поля користувач може змінювати для уточнення результатів

пошуку. Важливо також продумати механізм скидання фільтрів та повторного отримання початкового набору даних.

4. Повторення роботи з асинхронними запитами та обробкою

результатів. Для якісної підготовки до заняття необхідно повторити використання синтаксису `async/await` для виконання HTTP-запитів та обробки відповідей сервера. Особливу увагу слід приділити обробці помилок, перевірці результатів запиту та правильному перетворенню отриманих даних у динамічну HTML-розмітку списків. Також варто пригадати принцип додавання нових елементів у DOM за одну операцію з метою підвищення продуктивності.

Заняття 21. Розгортання backend-частини проєкту.

Актуальність та цілі заняття

Заняття 21. «Розгортання backend-частини

проєкту. Тренування з викладачем»

ХІД ЗАНЯТТЯ

Доброго дня!

На попередніх заняттях ви реалізували динамічні списки, пагінацію та фільтрацію даних у проєкті Flora, використовуючи HTTP-запити та роботу з DOM. Сьогодні ми робимо наступний логічний крок — підключаємо реальний бекенд і вчимося розгортати серверну частину проєкту для подальшої інтеграції з клієнтським застосунком.

Під час заняття ви налаштуєте локальне та віддалене середовище для запуску backend-частини, перевірите роботу API-ендпоінтів та підготуєте проєкт до повноцінної взаємодії з фронтендом. Особливу увагу буде приділено правильній організації структури запитів, обробці відповідей сервера та базовій діагностиці помилок під час звернення до API.

Це практичне заняття спрямоване на формування цілісного уявлення про взаємодію фронтенду з бекендом у реальному проєкті. Готуй свій проєкт — сьогодні ми створюємо фундамент для стабільної та масштабованої роботи застосунку.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Реалізація динамічних списків, фільтрації та пагінації неможлива без стабільного та коректно налаштованого серверного джерела даних. Розгортання backend-частини проєкту є обов'язковим етапом для переходу від навчальних запитів до реальної клієнт-серверної архітектури. Саме на цьому етапі формується розуміння повного циклу обробки даних — від запиту з інтерфейсу користувача до відповіді сервера та відображення результату на сторінці.

ЦІЛІ ЗАНЯТТЯ

Сформувати практичні навички розгортання backend-частини вебпроєкту та підготовки API до подальшого використання у фронтенд-частині застосунку.

ОСНОВНІ ЗАВДАННЯ

1. Навчитись розгортати backend-частину проєкту та підключати її до фронтенду згідно з вимогами макету та логіки сайту.
2. Перевірити коректність роботи основних API-ендпоінтів, що використовуються для формування списків, пагінації та фільтрації у проєкті.

Підготовка до заняття

1. Планування розробки бекенду та його структури

Backend-частина проєкту, яку ми розробимо, створивши репозиторій `fireplace_backend`, має реалізувати серверну логіку, яка обслуговує динамічні запити клієнта (фронтенду) і повертає дані для формування списків, пагінації і фільтрації. Цей бекенд має відповідати за збереження, обробку та доставку даних у форматі JSON через REST API, що дозволяє фронтенд-частині застосунку динамічно оновлювати розмітку у браузері.

Архітектура: сервер потрібно реалізувати як REST API, що надає ендпоінти для отримання даних (наприклад, колекції зображень, продуктів або іншого контенту) із параметрами фільтрації та пагінації.

Основні компоненти бекенду мають включати:

- Серверну оболонку (Express/FastAPI/Django REST Framework), яка приймає HTTP-запити;

- Контролери або маршрути, що відповідають за обробку GET/POST/PUT/DELETE-запитів для різних ресурсів;

- Логіку пагінації та фільтрації, яка приймає параметри `page`, `limit`, `query` тощо і повертає відповідні підмножини даних;

- Схему даних або моделі, які описують структуру зберігання даних (наприклад, через ORM або підключення до бази даних).

Обробку помилок та валідацію параметрів запиту для забезпечення коректних відповідей клієнту.

2. Створення та налаштування backend-проєкту

Створи репозиторій для backend-частини проєкту та його структуру й базові налаштування.

Для того, щоб пригадати як налаштувати робоче середовище, використовуй теми 1-6 дисципліни «[Основи Node.js. ІПЗ](#)».

Перед початком практичної роботи необхідно підготувати локальне середовище та розібратися з механікою запуску серверної частини проєкту.

Перевірте наявність усіх необхідних інструментів (Node.js та менеджера пакетів) і переконайтеся, що сервер успішно запускається без помилок.

Для цього слід проаналізувати конфігураційні файли, залежності та сценарії запуску сервера, виконавши наведені нижче кроки.

Інсталяція залежностей: перед запуском сервера необхідно встановити всі залежності (зазвичай через `npm install` для Node.js або `pip install -r requirements.txt` для Python).

Налаштування середовища: конфігураційні параметри (порт, URI бази даних, ключі тощо) зберігаються у файл `.env` або подібний конфігураційний файл.

Запуск сервера: сервер можна запустити локально через команду на кшталт `npm start`, `uvicorn main:app --reload` (для FastAPI) або відповідний скрипт запуску.

Тестування ендпоінтів: після запуску сервер слухає запити на певному порту (наприклад, `http://localhost:3000/api/...`), і фронтенд може робити до нього запити з параметрами для пагінації та фільтрації.

Підключення до фронтенду: Frontend робить асинхронні запити (`axios`, `fetch`) до бекенду з передачею параметрів сторінки та пошукових запитів. Backend обробляє ці параметри, виконує відповідні запити до джерела даних (файлова колекція, база даних) і повертає JSON-відповідь, яку фронтенд використовує для формування динамічної розмітки (списків, галерей тощо). Приклад відповіді API:

```
{  
  "data": [...],  
  "total": 150,  
  "page": 2,  
  "perPage": 15
```

Заняття 22. Розгортання backend-частини проєкту.

Актуальність та цілі заняття

Заняття 22. «Розгортання backend-частини проєкту. Практика»

ХІД ЗАНЯТТЯ

На цьому занятті ми переходимо до практичного розгортання серверної частини вашого проєкту Flora. Після ознайомлення з архітектурою backend-проєкту на попередньому занятті сьогодні ви зосередитесь на створенні базової структури серверного застосунку, налаштуванні середовища виконання, підключенні основних middleware та організації маршрутизації.

Під час заняття ви поетапно розгорнете backend-проєкт локально, перевірите коректність його запуску, налаштуєте конфігурації середовища та створите каркас API-маршрутів. Основний акцент буде зроблено на правильній структурі проєкту та готовності серверної частини до подальшої реалізації бізнес-логіки і підключення фронтенду.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Без правильно розгорнутої та структурованої backend-частини неможливо перейти до побудови повноцінної клієнт-серверної архітектури проєкту. Саме на етапі практичного розгортання серверу формується основа стабільної роботи застосунку,

зрозумілої маршрутизації, коректної конфігурації середовища та можливості подальшого масштабування проєкту.

Це заняття дозволяє закріпити знання базових принципів організації серверного коду та підготувати інфраструктуру проєкту до реалізації API на наступних етапах.

ЦІЛІ ЗАНЯТТЯ

Сформувати практичні навички розгортання backend-проєкту, побудови базової структури серверної частини, налаштування маршрутів та основних конфігурацій середовища.

ОСНОВНІ ЗАВДАННЯ

1. Закріпити практичні навички розгортання backend-частини для вебпроєкту [макету](#) Flora.
2. Навчитися формувати структуру бекенд-проєкту та налаштовувати маршрути.
3. Розгорнути локально backend-проєкт для сайту Flora та підготувати базову структуру серверної частини.
4. Налаштувати основні конфігурації середовища, middleware та каркас маршрутизації.

Підготовка до заняття

Підготовка до заняття

Перед початком практичної роботи необхідно повторити загальні принципи побудови серверного застосунку на Node.js, зокрема поняття Express-серверу, middleware, маршрутизації та змінних середовища.

На даному етапі виконання свого проєкту необхідно виконати такі завдання:

1) Підготовка проєктного середовища та структури. Необхідно клонувати або створити репозиторій backend-частини проєкту та підготувати базову структуру директорій відповідно до прийнятої архітектури (app, routes, controllers, middlewares, helpers, configs або аналогічні каталоги). Структура має бути зрозумілою, логічною та готовою до подальшого масштабування.

2) Налаштування середовища виконання. Потрібно перевірити наявність встановленого Node.js та менеджера пакетів, встановити всі залежності проєкту та налаштувати файл змінних середовища (.env). У конфігураціях має бути визначений порт сервера та інші базові параметри запуску.

3) Ініціалізація серверу. Необхідно створити або налаштувати головну точку запуску серверу (server.js або index.js), підключити основний файл застосунку та переконатися, що сервер коректно запускається локально без помилок.

4) Підключення middleware. У застосунку мають бути підключені базові middleware для обробки JSON-даних, CORS та логування (за потреби). Також має бути налаштований базовий обробник помилок.

5) Створення каркасу маршрутизації. Потрібно створити структуру маршрутів API та підключити їх до основного застосунку. На цьому етапі достатньо реалізувати порожні або тестові маршрути-заглушки, які підтверджують коректну роботу маршрутизації.

6) Перевірка запуску серверу. Після налаштування структури та конфігурацій необхідно перевірити запуск серверу та доступність тестових маршрутів через браузер або REST-клієнт (наприклад, Postman або Thunder Client).

Заняття 23. Взаємодія з базою даних PostgreSQL.

Актуальність та цілі заняття

Заняття 23. «Взаємодія з базою даних

PostgreSQL. Тренування з викладачем»

ХІД ЗАНЯТТЯ

Сучасному вебзастосунку потрібне сховище даних, яке забезпечує швидкий і надійний доступ до інформації. На цьому занятті ми зосередимося на використанні PostgreSQL, однієї з найпотужніших і найпопулярніших реляційних баз даних. PostgreSQL відома своєю надійністю, багатим набором функцій та відповідністю стандартам SQL.

Ви вже ознайомилися з основами реляційних баз даних та SQL на попередньому курсі, тому тепер настав час застосувати ці знання у практичних завданнях з використанням Node.js. Для інтеграції з базою даних ми будемо використовувати Sequelize, ORM (Object-Relational Mapping), яка спрощує роботу з базами даних, дозволяючи взаємодіяти з ними за допомогою JavaScript-коду замість написання складних SQL-запитів.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

У сучасних вебзастосунках взаємодія з базами даних є критично важливою частиною. Sequelize, як потужний ORM (Object-Relational Mapping) для реляційних баз даних, таких як

PostgreSQL, значно спрощує цей процес. У цій темі ми розглянемо основи виконання запитів в *Sequelize*, що дозволить вам ефективно взаємодіяти з базою даних, виконувати CRUD (Create, Read, Update, Delete) операції та будувати складні запити.

ЦІЛІ ЗАНЯТТЯ

Дізнатись, як використовувати *Sequelize* для зручного та продуктивного маніпулювання даними, що є ключовим для побудови функціональних та надійних вебзастосунків.

ОСНОВНІ ЗАВДАННЯ

1. Навчитись підключати PostgreSQL до backend-проєкту за допомогою *Sequelize* та перевіряти коректність з'єднання з базою даних.
2. Навчитись створювати та налаштовувати моделі *Sequelize* відповідно до структури таблиць бази даних (типи полів, обмеження, значення за замовчуванням, унікальність).
3. Навчитись синхронізувати моделі з базою даних та керувати створенням, оновленням і видаленням таблиць у процесі розробки.
4. Закріпити навички виконання базових операцій роботи з даними через *Sequelize* (створення, читання, оновлення та видалення записів).

Підготовка до заняття

1. Налаштування бази даних

Sequelize — це ORM для роботи з такими реляційними базами даних як Postgres, MySQL, MariaDB, SQLite і MSSQL. На сьогоднішній день це найпопулярніше рішення для [Node.js](#).

Підключення:

```
yarn add sequelize  
# або  
npm i sequelize
```

Підключення до БД:

```
const { Sequelize } = require('sequelize')  
  
// Варіант 1: передача `URI` для підключення  
const sequelize = new Sequelize('sqlite::memory:') // для  
`sqlite`  
const sequelize = new  
Sequelize('postgres://user:pass@example.com:5432/dbname')  
// для `postgres`  
  
// Варіант 2: передача параметрів окремо  
const sequelize = new Sequelize({  
  dialect: 'sqlite',  
  storage: 'path/to/database.sqlite'  
})  
  
// Варіант 2: передача параметрів окремо (для інших  
діалектів)
```

```
const sequelize = new Sequelize('database', 'username',  
'password', {  
  host: 'localhost',  
  dialect: /* 'mysql' | 'mariadb' | 'postgres' | 'mssql' */  
  */  
})
```

Перевірка підключення:

```
try {  
  await sequelize.authenticate()  
  console.log('З'єднання з БД було успішно встановлено')  
} catch (e) {  
  console.log('Неможливо виконати підключення до БД: ',  
e)  
}
```

За замовчуванням після того, як встановлено з'єднання, воно залишається відкритим. Для його закриття слід викликати метод `sequelize.close()`.

Модель — це абстракція, що представляє таблицю в БД. Модель повідомляє Sequelize декілька речей про сутність (entity), яку вона представляє: назву таблиці; те, які стовпчики вона містить (і їхні типи даних) тощо.

У кожної моделі є назва. Ця назва не обов'язково має збігатися з назвою відповідної таблиці. Зазвичай, моделі іменуються в однині (наприклад, `User`), а таблиці — у множині (наприклад,

Users). Sequelize виконує плюралізацію (переведення значення з однини в множину) автоматично.

Моделі можуть визначатися **двома способами**:

Шляхом виклику `sequelize.define(modelName, attributes, options)`.

Шляхом розширення класу `Model` і виклику `init(attributes, options)`.

Після визначення, модель доступна через команду `sequelize.model` + назва моделі. В якості прикладу створимо модель `User` з полями `firstName` та `lastName`.

```
sequelize.define
const { Sequelize, DataTypes } = require('sequelize')
const sequelize = new Sequelize('sqlite::memory:')

const User = sequelize.define(
  'User',
  {
    // Тут визначаються атрибути моделі
    firstName: {
      type: DataTypes.STRING,
      allowNull: false,
    },
    lastName: {
      type: DataTypes.STRING,
      // allowNull за замовчуванням має значення true
    },
  },
  {
    // Тут визначаються інші налаштування моделі
  }
)
```

```
// `sequelize.define` повертає модель  
console.log(User === sequelize.models.User) // true
```

Розширення Model

```
const { Sequelize, DataTypes, Model } =  
require('sequelize')  
const sequelize = new Sequelize('sqlite::memory:')  
  
class User extends Model {}  
  
User.init(  
  {  
    // Тут визначаються атрибути моделі  
    firstName: {  
      type: DataTypes.STRING,  
      allowNull: false,  
    },  
    lastName: {  
      type: DataTypes.STRING,  
    },  
  },  
  {  
    // Тут визначаються інші налаштування моделі  
    sequelize, // Екземпляр підключення (обов'язково)  
    modelName: 'User', // Назва моделі (обов'язково)  
  }  
)  
  
console.log(User === sequelize.models.User) // true
```

Під капотом `sequelize.define` використовує `Model.init`.

Надалі ми будемо використовувати тільки *перший варіант*.

Автоматичну плюралізацію назви таблиці можна вимкнути за допомогою налаштування `freezeTableName`:

```
sequelize.define(  
  'User',  
  {  
    // ...  
  },  
  {  
    freezeTableName: true,  
  })
```

Або глобально:

```
const sequelize = new Sequelize('sqlite:memory:', {  
  define: {  
    freezeTableName: true,  
  },  
})
```

У цьому випадку таблиця називатиметься `User`.

Назва таблиці може визначатися в очевидному вигляді:

```
sequelize.define(  
  'User',  
  {  
    // ...  
  },  
  {  
    tableName: 'Employees',  
  })
```


У цьому випадку таблиця називатиметься `Employees`.

Синхронізація моделі з таблицею:

`User.sync()` — створює таблицю за відсутності (наявна таблиця залишається незмінною).

`User.sync({ force: true })` — видаляє наявну таблицю і створює нову.

`User.sync({ alter: true })` — приводить таблицю у відповідність до моделі.

Приклад:

```
// Повертається проміс
await User.sync({ force: true })
console.log('Таблиця для моделі `User` щойно наново була створена!')
```

Синхронізація всіх моделей:

```
await sequelize.sync({ force: true })
console.log('Усі моделі були успішно синхронізовані.')
```

Видалення таблиці:

```
await User.drop()
console.log('Таблицю `User` було видалено.')
```

Видалення всіх таблиць

```
await sequelize.drop()
console.log('Було видалено всі таблиці.')
```

Sequelize приймає налаштування `match` з регулярним виразом, що дає змогу визначати групу таблиць, які синхронізуються:

```
// Виконуємо синхронізацію тільки тих моделей, назви яких закінчуються на `_test`.  
await sequelize.sync({ force: true, match: /_test$/ })
```

Зверніть увагу, що замість синхронізації в продакшні слід використовувати міграції.

За замовчуванням Sequelize автоматично додає в створювану модель поля `createdAt` та `updatedAt` з типом `DataTypes.DATE`. Це можна змінити:

```
sequelize.define(  
  'User',  
  {  
    // ...  
  },  
  {  
    timestamps: false,  
  }  
)
```

Названі поля відключати та перейменувати можна окремо:

```
sequelize.define(  
  'User',  
  {  
    // ...  
  },  
  {  
    timestamps: false,  
    createdAt: 'created_at',  
    updatedAt: 'updated_at',  
  }  
)
```

```

    {
      timestamps: true,
      // Відключаємо `createdAt`
      createdAt: false,
      // Змінюємо назву `updatedAt`
      updatedAt: 'updateTimestamp',
    }
  )

```

Якщо для колонки визначається тільки тип даних, синтаксис визначення атрибута можна скоротити наступним чином:

```

// ДО
sequelize.define('User', {
  name: {
    type: DataTypes.STRING,
  },
})

```

```

// ПІСЛЯ
sequelize.define('User', {
  name: DataTypes.STRING,
})

```

За замовчуванням значенням колонки є `NULL`. Це можна змінити за допомогою налаштування `defaultValue` (визначивши «дефолтне» значення):

```

sequelize.define('User', {
  name: {
    type: DataTypes.STRING,
    defaultValue: 'John Smith',
  },
})

```

```
    },  
  })
```

Як дефолтні можуть використовуватися спеціальні значення:

```
sequelize.define('Foo', {  
  bar: {  
    type: DataTypes.DATE,  
    // Поточні дата і час, що визначаються в момент  
    створення  
    defaultValue: Sequelize.NOW,  
  },  
})
```

Заняття 24. Взаємодія з базою даних PostgreSQL.

Актуальність та цілі заняття

Заняття 24. «Взаємодія з базою даних PostgreSQL. Практика»

ХІД ЗАНЯТТЯ

Доброго дня!

На попередньому занятті ви підготували структуру backend-проєкту, налаштували базові конфігурації сервера та маршрути для роботи з колекцією букетів у проєкті Flora. Сьогодні ми переходимо до практичної інтеграції бази даних PostgreSQL і замінюємо файлове зберігання даних на повноцінну роботу з реляційною базою.

Протягом заняття ви налаштуєте підключення серверної частини до хмарної бази даних, створите таблицю bouquets та під'єднаєте її до застосунку за допомогою ORM Sequelize. Ми поетапно розглянемо, як описувати модель даних, перевіряти успішність підключення та коректно обробляти помилки з'єднання з базою.

Окремо розберемо реалізацію маршруту для оновлення статусу обраного букету та перевіримо, як коректно повертати відповіді сервера у випадку успішного оновлення або відсутності запису в базі.

Це заняття спрямоване на формування практичного розуміння роботи backend-частини з реальною базою даних у проєкті. Готуй свій проєкт — сьогодні ми робимо важливий крок від навчального зберігання даних до повноцінної серверної архітектури з PostgreSQL.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Використання бази даних PostgreSQL та ORM-бібліотеки Sequelize є стандартною практикою у сучасній backend-розробці, оскільки дозволяє забезпечити надійне зберігання даних, масштабованість проєкту та спрощує роботу з таблицями і запитами без використання сирого SQL.

ЦІЛІ ЗАНЯТТЯ

Підключення реляційної бази даних та перенесення CRUD-логіки роботи з букетами на рівень ORM.

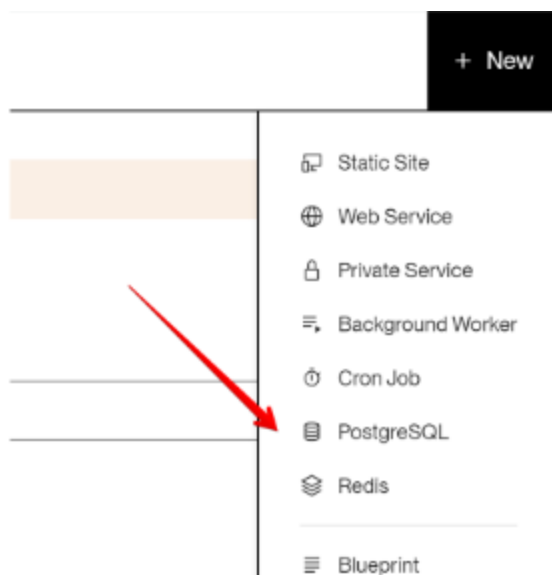
ОСНОВНІ ЗАВДАННЯ

1. Налаштувати підключення backend-проєкту до хмарної бази даних PostgreSQL за допомогою Sequelize.
2. Реалізувати роботу з таблицею bouquets у backend-частині проєкту [макету](#) Flora.

Підготовка до заняття **Підготовка до заняття**

На даному етапі виконання свого проєкту необхідно продовжити створення REST API для роботи з колекцією букетів (продуктів) та виконати такі завдання:

1) Створити акаунт на [Render](#). Після чого в акаунті створити нову базу даних PostgreSQL, яку треба назвати db-contacts:



2) Встанови графічний редактор [pgAdmin](#) для зручної роботи з базою даних для PostgreSQL. Підключися до створенної хмарної бази через графічний редактор та створи таблицю *bouquets*.

3) Використовуй вихідний код з попередніх практичних занять і заміни зберігання *bouquets* з json-файлу на створену тобою базу даних.

4) Напиши код для створення підключення до PostgreSQL за допомогою [Sequelize](#).

5) При успішному підключенні **виведи в консоль повідомлення "Database connection successful"**.

6) Обов'язково **оброби помилку підключення**. Виведи в консоль повідомлення помилки і заверши процес використовуючи `process.exit(1)`.

7) У функціях обробки запитів **заміни код CRUD-операцій** над продуктами (букетами) з файлу, на Sequelize-методи для роботи з колекцією `bouquets` в базі даних.

Sequelize-модель `bouquets`:

```
const Bouquet = sequelize.define(
  'contact', {
    photo: {
      type: DataTypes.STRING,
      allowNull: false,
    },
    title: {
      type: DataTypes.STRING,
      allowNull: false,
    },
    description: {
      type: DataTypes.STRING,
      allowNull: false,
    },
    favorite: {
      type: DataTypes.BOOLEAN,
      defaultValue: false,
    },
  },
  {}
)
```

8) В колекції з'явилося додаткове поле статусу `favorite`, яке приймає логічне значення `true` або `false`. Воно відповідає за те, що в обраному чи ні знаходиться зазначений букет. Потрібно реалізувати для оновлення статусу букету новий роутер:

PATCH /api/bouquets/:bouquetId/favorite

Отримує параметр `bouquetId`

Отримує `body` в json-форматі с оновленням поля `favorite`

Якщо з `body` все добре, викликає функцію

`updateStatusBouquet (bouquetId, body)` (напиши її) для поновлення продукту в базі

За результатом роботи функції повертає оновлений об'єкт букету і статусом `200`. В іншому випадку, повертає json з ключем `{"message": "Not found"}` і статусом `404`

9) Проєкт коректно працює з актуальною LTS-версією Node

Заняття 25. REST API. Тренування з викладачем.

Актуальність та цілі заняття

Заняття 25. «REST API. Тренування з викладачем»

ХІД ЗАНЯТТЯ

Доброго дня!

На попередніх заняттях ви вже розгорнули backend-частину проєкту та ознайомилися зі структурою серверного застосунку. Сьогодні ми зосереджуємося на практичному опрацюванні принципів REST та правильній організації API для проєкту.

Протягом заняття ви будете *аналізувати існуючі маршрути та контролери*, перевіряти відповідність ендпоінтів REST-підходу, а також навчитеся коректно поєднувати ресурси, HTTP-методи та структуру URL. Ми розберемо, як правильно формувати адреси ресурсів, як передавати параметри через query-рядок і як відокремлювати логіку маршрутизації від бізнес-логіки контролерів.

Особливу увагу приділимо *ролі middleware*, схем валідації та єдиному підходу до формування відповідей сервера. Ви побачите, як самоописові відповіді та чітка структура API значно спрощують інтеграцію фронтенду з бекендом.

Це заняття спрямоване на *формування практичного розуміння* того, як *будувати REST API у реальному проєкті* та як підготувати серверну частину до подальшої роботи з базою даних і складнішою бізнес-логікою. Готуй свій проєкт — сьогодні ми приводимо API проєкту до зрозумілої, стабільної та професійної структури.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Розуміння принципів REST та коректного проєктування API є необхідною умовою для побудови стабільної взаємодії між фронтендом і бекендом у проєкті, оскільки саме REST-підхід визначає структуру URL, правила роботи з ресурсами, формат відповідей сервера та логіку використання HTTP-методів.

ЦІЛІ ЗАНЯТТЯ

Сформувати цілісне практичне розуміння принципів REST API та навчитися коректно проєктувати ендпоінти, маршрути й контролери для взаємодії фронтенду з бекендом.

ОСНОВНІ ЗАВДАННЯ

1. Навчитись проєктувати REST-ендпоінти відповідно до архітектурних принципів REST.
2. Потренуватись правильно застосовувати HTTP-методи під час роботи з ресурсами проєкту.

Підготовка до заняття

1. Що таке REST

REST (REpresentational State Transfer) – розшифровується як передача репрезентативного стану. Це набір принципів проєктування для підвищення масштабованості та гнучкості мережових комунікацій.

Ці принципи відповідають на ряд запитань.

Які компоненти системи?

Як вони спілкуються між собою?

Як можна гарантувати можливість змінити різні частини системи у будь-який час?

Як можна масштабувати систему для обслуговування великої кількості користувачів?

Рой Т. Філдінг вперше ввів термін REST у 2000 році у своїй докторській дисертації «Архітектурні стилі та проектування мережових архітектур програмного забезпечення». Її можна почитати за цією [адресою](#).

На момент публікації дисертації Інтернет уже набув широкого поширення. Філдінг, по суті, «відступив на крок назад» і проаналізував характеристики, які зробили Інтернет успішнішим за конкуруючі мережові протоколи. Потім він представив схематичну структуру, яка зробить мережеве спілкування «вебподібним». Отже, REST – це загальний набір принципів, не специфічних для Інтернету. Його можна застосовувати до інших

типів мереж, наприклад, до вбудованих систем. REST також не є протоколом, оскільки не вимагає подробиць реалізації. У дисертації Філдінг викладає ряд архітектурних обмежень, яким повинна задовольняти система, щоб вважатися RESTful.

2. Що таке URI та URL

Перш ніж ми перейдемо до того, які архітектурні обмеження накладає REST, давайте розберемося з термінологією URL та URI. Терміни URI та URL часто використовуються як синоніми, але це не зовсім одне й те саме.

URI є ідентифікатором конкретного ресурсу. Як сторінку, книгу або документ.

URL представляє особливий тип ідентифікатора, який також говорить вам, як отримати до нього доступ.

В принципі URI – це ширше поняття і містить URL. Якщо проводити якісь аналогії спрощення, то можна вважати, що URI – це щось на зразок імені, а URL – конкретно ім'я і як до нього дістатися. Наприклад, ISBN книги – це URI, а ось <https://goit.ua> – це URL тут є ім'я і як до нього дійти – протокол. Тобто, якщо протокол (https, ftp тощо) або присутній, або мається на увазі для домену, ви повинні називати його URL-адресою, навіть якщо це також URI.

Існує наступна традиційна форма запису URL:

```
<scheme>:// [<login>[:<password>]@]<host>[:<port>]] [/<path>] [/?<query>] [#<fragment>]
```

Приклад:

```
http://user:password@host.com:80/resource/path/?query=name&ttt=123#hash
```

У цьому записі:

scheme – це мережевий протокол, за яким відбувається звернення до ресурсу

login – не обов'язковий параметр ім'я користувача, яке використовується для доступу до ресурсу

password – пароль вказаного користувача, якщо він необхідний

host – повністю прописане доменне ім'я хоста в системі DNS (goit.ua) або IP-адреса хоста у формі чотирьох груп десяткових чисел, розділених крапками

port – порт хоста для підключення, згадайте ми часто використовували <http://localhost:3000>, де 3000 – це і є порт.

Виникає закономірне питання, чому порт не вказується наприклад для URL у браузері. Просто браузер знає, що для протоколу `http` порт дорівнює 80, а для `https` він дорівнює 443 і підставляє його за нас

path – уточнююча інформація про місцезнаходження ресурсу.

query – рядок запиту з переданими на сервер (методом GET) параметрами. Починається із символу `?`, роздільник параметрів – знак `&`. Приклад: `?foo=123&baz=234&bar=value`

fragment – ідентифікатор із попереднім символом #. Часто його називають хеш – hash або якір. Якорем може бути вказаний заголовок всередині документа або атрибут id елемента. За таким посиланням браузер відкриє сторінку та перемістить вікно перегляду вказаного елемента.

Наприклад, посилання на секцію контактів у лендінгу:

<https://example.com/#contact>.

Заняття 26. REST API. Практика.

Актуальність та цілі заняття

Заняття 26. «REST API. Практика»

ХІД ЗАНЯТТЯ

Доброго дня!

На попередніх заняттях ви ознайомилися з принципами побудови REST API та структурою backend-проєкту. Сьогодні ми зосередимося на практичній реалізації повноцінного REST API для роботи з колекцією букетів у проєкті Flora.

Під час заняття ви будете реалізовувати маршрути для отримання, створення, оновлення та видалення ресурсів, навчитеся відокремлювати бізнес-логіку у сервісний шар та правильно організовувати контролери.

Особливу увагу приділимо валідації вхідних даних за допомогою схем, обробці помилок та формуванню коректних HTTP-відповідей відповідно до вимог REST.

У процесі роботи ви перевірите створені ендпоінти за допомогою клієнта для тестування API та відпрацюєте типові сценарії взаємодії клієнта з сервером.

Це заняття спрямоване на закріплення навичок побудови структури REST-сервісу, підготовку backend-частини проєкту Flora

до подальшого підключення складнішої бізнес-логіки та інтеграції з іншими частинами системи.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Розуміння принципів побудови REST API та вміння реалізовувати стандартні CRUD-операції є базовою вимогою для розробки сучасних серверних застосунків і подальшої інтеграції бекенду з клієнтською частиною.

ЦІЛІ ЗАНЯТТЯ

Сформувані практичні навички реалізації REST API для роботи з ресурсами у backend-проєкті з використанням сервісного шару, валідації даних та коректної обробки помилок.

ОСНОВНІ ЗАВДАННЯ

1. Реалізувати повний набір REST-ендпоінтів для роботи з колекцією букетів.
2. Відпрацювати побудову контролерів, сервісів і схем валідації у проєкті [макету](#) Flora.

Підготовка до заняття

Підготовка до заняття

Перед початком практичної роботи необхідно повторити логіку взаємодії фронтенду з REST API та підготувати власний backend-проект до запуску.

Рекомендується звернути увагу на такі аспекти:

- принцип роботи HTTP-запитів з параметрами page, limit, search;
- формат відповіді сервера для побудови динамічних списків;
- обробку помилок запиту на стороні клієнта.

На даному етапі виконання свого проєкту необхідно написати REST API для роботи з колекцією букетів. Для роботи з REST API використовуй [\[Postman\]](#). Для цього виконати такі завдання:

- Встанови модулі командою `npm i`
- У файл `bouquetServices.js` (знаходиться в папці `services`) напиши функції.
- Напиши контролери у файлі `bouquetsControllers.js` (знаходиться у папці `controllers`) з урахуванням вимог (дивись нижче)

REST API повинен підтримувати такі раути:

GET /api/bouquets

Викликає функцію-сервіс `listBouquets` для роботи з json-файлом `bouquets.json`

Повертає масив всіх букетів в json-форматі зі статусом `200`

GET /api/bouquets/:id

Викликає функцію-сервіс `getBouquetById` для роботи з json-файлом `bouquets.json`

Якщо букет за `id` знайдений, повертає об'єкт букету в json-форматі зі статусом `200`

Якщо букет за `id` не знайдено, повертає json формату `{"message": "Not found"}` зі статусом `404`

DELETE /api/bouquets/:id

Викликає функцію-сервіс `removeBouquets` для роботи з json-файлом `bouquets.json`

Якщо букет за `id` знайдений і видалений, повертає об'єкт видаленого букету в json-форматі зі статусом `200`

Якщо букет за `id` не знайдено, повертає json формату `{"message": "Not found"}` зі статусом `404`

POST /api/bouquets

Отримує `body` в json-форматі з полями `{photo, title, description, price, favourite}`. Усі поля є обов'язковими - для валідації створи у файлі `bouquetsSchemas.js` (знаходиться у папці `schemas`) схему з використанням пакета `joi`

Якщо в `body` немає якихось обов'язкових полів (або передані поля мають не валідне значення), повертає json формату

`{"message": error.message}` (де `error.message` — змістовне повідомлення з суттю помилки) зі статусом `400`

Якщо `body` валідне, викликає функцію-сервіс `addBouquet` для роботи з json-файлом `bouquets.json`, з передачею їй даних з `body`

За результатом роботи функції повертає новостворений об'єкт з полями `{id, photo, title, description, price, favourite}` і статусом `201`

PUT /api/bouquets/id

Отримує `body` в json-форматі з будь-яким набором оновлених полів (`photo, title, description, price, favourite`) (всі поля вимагати в боді як обов'язкові не потрібно: якщо якийсь із полів не передане, воно має зберегтись у букеті зі значенням, яке було до оновлення).

Якщо запит на оновлення надійшов без передачі в `body` принаймні одного поля, сервер повертає JSON у форматі `{"message": "Body must have at least one field"}` зі статусом `400` (Bad Request).

Передані в боді поля мають бути провалідовані. Для валідації створи у файлі `bouquetsSchemas.js` (знаходиться у папці `schemas`) схему з використанням пакета `Joi`. Якщо передані поля мають не валідне значення, повертає JSON у форматі `{"message": error.message}` (де `error.message` — змістовне повідомлення з суттю помилки) зі статусом `400` (Bad Request).

Якщо з `body` все добре, викликає функцію-сервіс `updateBouquet`, яку слід створити в файлі `bouquetsServices.js` (знаходиться в папці `services`). Ця функція має приймати `id` букету, що підлягає оновленню, та дані з `body`, і оновити букет у json-файлі `bouquets.json`

За результатом роботи функції повертає оновлений об'єкт букету зі статусом `200`.

Якщо букет за `id` не знайдено, повертає JSON у форматі `{"message": "Not found"}` зі статусом `404`

Зверни увагу! Валідацію `body` можна як здійснювати у контролері, так і створити для цих цілей окрему міddlвару, яка буде викликатись до контролера. Для створення міddlвари можеш скористатись функцією `validateBody.js` у папці `helpers`

Для роботи з помилками можна скористатись функцією `HttpError.js` у папці `helpers`.

Якщо вказані функції використовувати не будеш, видали їх з проєкту перед тим, як надсилатимеш роботу на перевірку ментору.

Заняття 27. Робота з файлами. Зображення.

Актуальність та цілі заняття

Заняття 27. «Робота з файлами. Зображення.

Тренування з викладачем»

ХІД ЗАНЯТТЯ

Доброго дня!

На попередніх заняттях ви вже навчилися працювати з REST API та підключати фронтенд-частину проєкту Flora до серверної логіки. Сьогодні ми переходимо до ще одного важливого практичного етапу — реалізації завантаження файлів і зображень у backend-частині застосунку.

Протягом заняття ми *налаштуємо прийом файлів із клієнтської частини* за допомогою форм із типом кодування multipart/form-data, підключимо middleware Multer та навчимося обробляти завантажені зображення на сервері. Ми покроково *розберемо, як зчитувати файл із запиту*, як зберігати його у тимчасову директорію, переміщувати у папку постійного зберігання та формувати коректну відповідь сервера.

Окрему увагу приділимо *структурі backend-проєкту Flora* під час роботи з файлами: організації папок для збереження зображень, обробці помилок завантаження, обмеженню розміру файлів та перевірці наявності файлу в запиті. Ви побачите, як правильно

поєднувати обробку файлів із уже реалізованими контролерами та сервісами.

Це заняття спрямоване на *відпрацювання роботи із зображеннями у вебпроєктах* — від надсилання файлу з форми до його збереження на сервері та подальшого використання у проєкті Flora. Сьогодні ми закладаємо основу для повноцінної роботи з медіаконтентом у вашому backend-застосунку.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Робота з файлами та зображеннями є типовою задачею для сучасних вебзастосунків, тому вміння приймати, перевіряти, зберігати та обробляти файли на стороні сервера є необхідним елементом повноцінної backend-розробки.

ЦІЛІ ЗАНЯТТЯ

Сформувані практичні навички реалізації завантаження файлів на сервер, обробки multipart/form-data та збереження зображень у backend-проєкті.

ОСНОВНІ ЗАВДАННЯ

1. Навчитись реалізовувати прийом файлів із клієнтської частини та зберігати зображення на сервері з використанням middleware Multer.

2. Налаштувати обробку multipart/form-data та коректну обробку помилок під час завантаження файлів.
3. Реалізувати базову логіку збереження та переміщення файлів у проєкті.

Підготовка до заняття

1. Завантаження файлів

HTML форма, надіслана на сервер браузером, повинна бути закодована. Якщо ми не вказуємо метод кодування, то за замовчуванням використовується `application/x-www-form-urlencoded`. Ми вже стикалися з цим методом кодування, коли розглядали передачу даних на сервер та знаємо, що він підтримується Express.

Але якщо виникає потреба завантажувати файли на сервер, завдання стає трохи складнішим. Нам необхідно для надсилання файлів застосовувати тип кодування `multipart/form-data`, який, як ви зрозуміли, не обробляється Express напямую.

Існує декілька пакетів для завантаження файлів на сервер. Ми розглянемо роботу з двома найпопулярнішими пакетами [Multer](#) та [Formidable](#). Обидва пакети мають переваги та недоліки, але рекомендація буде використовувати пакет Multer як новіший.

Перш ніж перейти до прикладів використання цих пакетів, уточнимо одне загальне питання: що робити після обробки форми? Після того як форму опрацьовано, потрібно повернути відповідь браузеру. Існують два поширені підходи:

Перший — пряма відповідь. Після обробки форми ми можемо надіслати HTML або JSON безпосередньо браузеру або клієнту.

Другий підхід — це переадресація. Він популярніший за перший і використовує здебільшого два коди відповіді. Перший код відповіді – 302 (Found — Знайдено) та другий код відповіді – 303 (See Other — Дивитися інше), задіяти який, на наш погляд, краще. Express за замовчуванням використовує для статусу код відповіді 302.

2. Multer

Multer — це проміжне ПЗ (middleware) фреймворка Express, яка використовується при завантаженні файлів для обробки `multipart/form-data`.

По суті є обгорткою над низькорівневим пакетом `busboy`, що дозволяє використовувати його максимально ефективно. Multer не обробляє жодний інший тип форм, крім `multipart/form-data`.

Встановлення:

```
npm install -S multer
```

Multer додає об'єкт `body` та об'єкт `file` (або `files`) всередину об'єкта `request`. Об'єкт `body` містить значення текстових полів форми, об'єкт `file` (`files`) містить файл або файли, що завантажуються через форму. Повний код нашого застосунку наступний.

```
const createError = require('http-errors');
const express = require('express');
const path = require('path');
const fs = require('fs').promises;
const app = express();
const multer = require('multer');
const uploadDir = path.join(process.cwd(),
'uploads');
const storeImage = path.join(process.cwd(),
'images');

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, uploadDir);
  },
  filename: (req, file, cb) => {
    cb(null, file.originalname);
  },
  limits: {
    fileSize: 1048576,
  },
});

const upload = multer({
  storage: storage,
});

app.post('/upload', upload.single('picture'), async
(req, res, next) => {
  const { description } = req.body;
```

```

    const { path: temporaryName, originalname } =
req.file;
    const fileName = path.join(storeImage,
originalname);
    try {
        await fs.rename(temporaryName, fileName);
    } catch (err) {
        await fs.unlink(temporaryName);
        return next(err);
    }
    res.json({ description, message: 'Файл успішно
завантажено', status: 200 });
    });

    // catch 404 and forward to error handler
    app.use((req, res, next) => {
        next(createError(404));
    });

    app.use((err, req, res, next) => {
        res.status(err.status || 500);
        res.json({ message: err.message, status: err.status
});
    });

    const isAccessible = path => {
        return fs
            .access(path)
            .then(() => true)
            .catch(() => false);
    };

    const createFolderIsNotExist = async folder => {
        if (!(await isAccessible(folder))) {
            await fs.mkdir(folder);
        }
    }

```

```
};
```

```
const PORT = process.env.PORT || 3000;
```

```
app.listen(PORT, async () => {  
  createFolderIsNotExist(uploadDir);  
  createFolderIsNotExist(storeImage);  
  console.log(`Server running. Use on port:${PORT}`);  
});
```

Цей приклад демонструє типовий підхід до завантаження файлів у Express, тому зосередимося на ключових особливостях використання Multer. Пакет надає наступну інформацію щодо кожного завантажуваного файлу:

Ключ	Опис	Зауваження
<code>fieldname</code>	Ім'я поля, задане у формі	
<code>originalname</code>	Ім'я файлу на комп'ютері користувача	
<code>encoding</code>	Кодування файлу	
<code>mimetype</code>	Mime-тип файлу	
<code>size</code>	Розмір файлу в байтах	
<code>destination</code>	Каталог, де буде збережено файл	<code>DiskStorage</code>
<code>filename</code>	Ім'я файлу без <code>destination</code>	<code>DiskStorage</code>
<code>path</code>	Повний шлях до завантажуваного файлу	<code>DiskStorage</code>
<code>buffer</code>	<code>Buffer</code> з усього файлу	<code>MemoryStorage</code>

Створюємо двигун дискового простору `DiskStorage`

```
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, uploadDir);
  },
  filename: (req, file, cb) => {
    cb(null, file.originalname);
  },
  limits: {
    fileSize: 1048576,
  },
});
```

За допомогою опцій, розташування `destination` та ім'я файлу `filename` ми визначаємо, де буде знаходитись файл після завантаження.

`destination` використовується, щоб задати каталог, в якому будуть розміщені файли.

`filename` використовується, щоб визначити, як буде названо файл всередині каталогу.

У параметрах кожної функції є запит (`req`) та набір інформації про файл (`file`, дивись попередню таблицю)

Також присутній об'єкт `limits`, що встановлює обмеження. Він повністю збігається з методами пакету `busboy`. Переглянути повний [список методів](#). В нашому прикладі ми встановили максимальний розмір файлу в 1 Мбайт.

Ми створюємо екземпляр `multer`

```
const upload = multer({  
  storage: storage,  
});
```

Далі ми використовуємо проміжне ПЗ

```
upload.single('picture');
```

Воно завантажує один файл з ім'ям `picture` у тимчасову папку `uploads`, а інформація про файл буде збережена в `req.file`

Всередині обробника ми переносимо файл до папки постійного зберігання `images`

Повний код прикладу на [Github Gist](#)

Заняття 28. Робота з файлами. Зображення.

Актуальність та цілі заняття

Заняття 28. «Робота з файлами. Зображення.

Практика»

ХІД ЗАНЯТТЯ

Доброго дня!

На попередньому занятті ви ознайомилися з принципами завантаження файлів у backend-застосунку та роботою з middleware для обробки multipart-запитів. Сьогодні ми переходимо до повноцінної практики та зосереджуємося на реалізації роботи із зображеннями у проєкті Flora.

Під час заняття ви *налаштуєте прийом файлів із клієнтської частини*, організуєте тимчасове та постійне зберігання зображень, а також *реалізуєте оновлення фото для букетів через окремий ендпоінт*. Ми детально розберемо, як *правильно формувати шлях до файлу*, як створювати публічні URL для зображень і як зберігати ці посилання у базі даних.

Окрему увагу буде приділено *роздачі статичних файлів через Express*, перевірці авторизації під час завантаження файлів та обробці типових помилок (відсутність файлу, некоректний формат, помилки доступу до файлової системи).

Це заняття спрямоване на *відпрацювання реального сценарію роботи з медіаконтентом у вебпроєктах* — від завантаження файлу з клієнта до формування коректного публічного посилання на зображення у backend-частині застосунку Flora.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

У більшості сучасних вебпроєктів робота з файлами та зображеннями є базовою вимогою, тому вміння правильно реалізувати завантаження, зберігання та роздачу медіафайлів є необхідною навичкою для backend-розробника.

ЦІЛІ ЗАНЯТТЯ

Навчитися реалізовувати повний цикл роботи із зображеннями в backend-частині проєкту Flora: від прийому файлу з клієнта до збереження його на сервері та формування публічного URL.

ОСНОВНІ ЗАВДАННЯ

1. Реалізувати завантаження та оновлення зображень для букетів у backend-частині проєкту [макету](#) Flora.
2. Налаштувати роздачу статичних файлів і збереження посилань на зображення в базі даних.

Підготовка до заняття **Підготовка до заняття**

На даному етапі виконання свого проєкту необхідно виконати такі завдання:

1) Продовж створення REST API для роботи з колекцією ,ertnsd. Додай можливість завантаження аватарки користувача через [Multer](#) .

2) Створи папку `public` для роздачі статички. У цій папці створи підпапку `photos`.

3) Налаштуй Express на роздачу статичних файлів з папки `public`.

4) Поклади будь-яке зображення в папку `public/photos` і перевір, що роздача статички працює.

5) Під час переходу за таким URL браузер має відобразити зображення аватара:

```
http://localhost:<порт>/avatars/<ім'я_файлу_з_розширенням>
```

Приклад: <http://localhost:3000/avatars/user-123.png>

6) У схему користувача додай нову властивість `photoURL` для зберігання зображення.

```
{  
  ...  
  photoURL: DataTypes.STRING,  
  ...  
}
```

Використовуй пакет [gravatar](#) для того, щоб при додаванні нового букету відразу згенерувати йому фото по його `id`.

7) При додаванні букету створюй посилання на фото за допомогою [gravatar](#) та отриманий URL збережи в поле `photoURL` під час створення букету.

8) Додай можливість поновлення інформації про букет, створивши ендпоінт `/auth/photos` і використовуючи метод `PATCH`.

```
# Запит
```

```
PATCH /auth/зрещі
```

```
Content-Type: multipart/form-data
```

```
Authorization: "Bearer {{token}}"
```

```
RequestBody: завантажений файл
```

```
# Успішна відповідь
```

```
Status: 200 OK
```

```
Content-Type: application/json
```

```
ResponseBody: {
```

```
  "avatarURL": "тут буде посилання на зображення"
```

```
}
```

```
# Неуспішна відповідь
```

```
Status: 401 Unauthorized
```

```
Content-Type: application/json
```

```
ResponseBody: {
```

```
  "message": "Not authorized"
```

```
}
```


9) Створи папку `temp` в корені проєкту і зберігай в неї завантажене фото.

10) Перенеси фото букету з папки `temp` в папку `public/photos` і дай їй унікальне ім'я для конкретного букету.

11) Отриманий URL `/photos/<ім'я файлу з розширенням>` та збережи в поле `photoURL` букету.

12) Додаткове завдання (необов'язкове). Unit-тести для контролера (Jest)

Напиши unit-тести для контролера за допомогою Jest і перевір у тестах, що:

- відповідь має статус-код 200;
- у відповіді повертається токен;
- у відповіді повертається об'єкт `bouquet` з полями `photo`, `title`, `description`, `price`, і кожне з цих полів має тип даних `String`.

Заняття 29. Хостинг, документація swagger.

Актуальність та цілі заняття

Заняття 29. «Хостинг та документація swagger.

Тренування з викладачем»

ХІД ЗАНЯТТЯ

Доброго дня!

На попередніх заняттях ви реалізували серверну частину проєкту, роботу з REST API, завантаження файлів та зображень, а також підготовку бекенду до інтеграції з клієнтським застосунком.

Сьогодні ми зосереджуємося на двох ключових етапах підготовки проєкту до реального використання — розгортанні бекенду на хостингу та створенні повноцінної інтерактивної документації API за допомогою Swagger.

Протягом заняття ви розгорнете backend-проєкт у віддаленому середовищі, перевірите роботу всіх ендпоінтів у продакшн-режимі, а також навчитесь підключати та налаштовувати Swagger UI для автоматизованої документації REST API.

Особливу увагу приділимо опису маршрутів, параметрів запитів, схем body та форматів відповідей, а також використанню Swagger як інструмента тестування API під час командної роботи.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Уміння розгортати серверний застосунок на хостингу та підтримувати актуальну API-документацію є обов'язковими вимогами до сучасного backend-проєкту та командної розробки.

ЦІЛІ ЗАНЯТТЯ

Сформувати навички публічного розгортання backend-проєкту та створення інтерактивної документації REST API за допомогою Swagger.

ОСНОВНІ ЗАВДАННЯ

1. Навчитись розгортати backend-частину проєкту на хостингу.
2. Підключити Swagger UI до проєкту та налаштувати документацію API.
3. Описати всі основні ендпоінти, параметри та формати відповідей.
4. Перевірити роботу ендпоінтів через інтерфейс Swagger.

Підготовка до заняття

Підготовка до заняття

Перед початком заняття необхідно:

переконавшись, що backend-проєкт коректно запускається локально;

перевірити роботу всіх реалізованих ендпоінтів (CRUD, завантаження зображень, авторизація);
підготувати змінні середовища для запуску проєкту на віддаленому сервері.

1. API-документація Swagger

Swagger використовується для опису та візуалізації REST API. Файл `swagger.json` (або `swagger.yaml`) містить повний опис сервера:

- список усіх ендпоінтів;
- HTTP-методи;
- параметри запитів (`params`, `query`, `headers`);
- структури `body`;
- формати відповідей та коди статусів.

Після підключення Swagger UI ви отримуєте інтерактивну сторінку документації, з якої можна:

- переглядати всі маршрути API;
- тестувати запити без використання Postman;
- перевіряти коректність структури відповідей.

На занятті ви:

- підключите Swagger UI до Express-проєкту;
- створите базовий `swagger`-файл;
- опишете ендпоінти колекції букетів та роботу із зображеннями.

Таким чином, файл `swagger.json` міститиме відомості про всі ендпоінти: параметри, схеми запитів і відповідей. Це дозволяє автоматично згенерувати красиву, інтерактивну документацію

для серверу. Проте, **swagger.json** ми розробимо на наступних заняттях.

Ну а поки спроектуємо ендпоінти для отримання всіх товарів, одного товару, здійснення замовлення та отримання фідбеків. А також опишемо схеми. Отримаємо такий вигляд документації:

Fireplace Fireplaces endpoints ^	
GET	/fireplaces Get fireplace list v
GET	/fireplaces/{id} Get fireplace element by ID v
Order Orders endpoints ^	
POST	/orders Create user's order v
Feedback Feedbacks endpoints ^	
GET	/feedbacks Feedbacks list v
Schemas ^	
GetFireplaceResponse >	
GetFireplaceByIdResponse >	
CreateOrderRequest >	
CreateOrderResponse >	
GetFeedbacksResponse >	

GET

/fireplaces

Get fireplace list

^

Parameters

Try it out

Name	Description
page integer (query)	Number of the portion requesting data. Default: 10. (min: 1) Default value : 1
limit integer (query)	Amount of the data in portion requesting. (min: 1 - max: 30) Default value : 10
title string (query)	Searching value string (min: 2 - max: 32 characters). Example: Антикварна Арка
type string (query)	Type of fireplace category 'top' 'standart' Available values : top, standart
sortOrder string (query)	The order sorting items by title - asc: a -> z, desc: z -> a Available values : asc, desc

Responses

Code	Description	Links
200	Successfull operation	No links
	Media type application/json Controls Accept header. Example Value Schema <pre>{ "fireplace": [{ "_id": "685183fb41a1c6bc3279d2da", "title": "Острівний Дуб", "description": "Витончений заміський портал із добірного масиву дуба, що створює атмосферу унікального острівного спокою у вашому домі.", "img": "https://ftp.goit.study/img/Fireplace/124925136a1003120835.png", "price": 37900, "category": "standart" }], "totalItems": 76, "page": 1, "limit": 10 }</pre>	
400	Bad request (invalid request params)	No links
	Media type	

GET

/fireplaces/{id} Get fireplace element by ID

^

Parameters

Try it out

Name	Description
id	Valid fireplace ID
any (path)	id

Responses

Code	Description	Links
200	Successfull operation	No links
Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": "685183fb41a1c6bc3279d2da", "title": "Острівний Дуб", "description": "Витончений замський портал із добірного масиву дуба, що створює атмосферу унікального острівного спокою у вашому домі.", "fullDescription": "Камінний портал «Острівний Дуб» виконаний з добірного масиву дуба, обробленого таким чином, щоб підкреслити його природну текстуру та річні кільця. Його теплий дерев'яний відтінок та класичні форми створюють атмосферу затишного замського будинку або острівного котеджу, де панує спокій та розміреність. Ідеально поєднується з інтер'єрами в стилі кантрі, рустік або класика.", "img": "https://ftp.goit.study/img/fireplace/124925136a1003120835.png", "price": 37900, "category": "standart"}</pre>		
400	Bad request (invalid request params)	No links
404	Fireplace not found	No links
Media type		

Order

Orders endpoints

^

POST

/orders Create user's order

^

Parameters

Try it out

No parameters

Request body

required

application/json

Example Value | Schema

```
{  "name": "Максим",  "phone": "380991232343",  "address": "вул. Гетьмана Мазепи 15, Львів Error",  "message": "Доброго дня, хочу замовити піч...",  "modelId": "685183fb41a1c6bc3279d2da"}
```

Responses

Code	Description	Links
200	Successfull operation	No links
Media type application/json Controls Accept header. Example Value Schema <pre>{ "_id": "685183fb41a1c6bc3279d2aa", "name": "Максим", "phone": "380991232343", "address": "вул. Гетьмана Мазепи 15, Львів Error", "message": "Доброго дня, хочу замовити піч...", "model": "Лофт Кортен", "orderNum": "Q007G19ZCY"}</pre>		
400	Bad request (invalid request params)	No links
Media type		



GET /feedbacks Feedbacks list



Parameters Try it out

Name	Description
limit integer (query)	Number of returned feedbacks in list. (min: 3 - max: 25) Default value : 10
page integer (query)	Number of feedbacks in returned data relative to limit parameter Default value : 1

Responses

Code	Description	Links
200	Successfull operation	No links
	Media type application/json Controls Accept header. Example Value Schema <pre>{ "feedbacks": [{ "id": "675a1b2c41a1c6bc3279f001", "name": "Максим Задорожний", "descr": "Придбав Хліб Коло для сімейної вітальні. Кругла форма топки стала справ...", "rate": 5, "city": "м.Львів" }], "total": 28, "page": 1, "limit": 10 }</pre>	
400	Bad request (invalid request params)	No links
	Media type	

Заняття 30. Хостинг, документація swagger.

Актуальність та цілі заняття

Заняття 30. «Хостинг та документація swagger.

Рев'ю коду. Деплой проєкту. Практика»

ХІД ЗАНЯТТЯ

Доброго дня! На попередньому занятті ви ознайомилися з принципами створення та підключення Swagger-документації до backend-проєкту, а також навчилися описувати базові ендпоінти та структури відповідей.

Сьогодні ми *переходимо до фінальної практики* та зосереджуємося на *завершенні та перевірці* Swagger-документації для проєкту Flora.

Під час заняття ви *допрацюєте всі схеми в розділі components → schemas, приведете їх у відповідність до Joi-валідацій і моделей бази даних, а також перевірите правильність опису requestBody та responses для всіх ендпоінтів.*

Окрему увагу буде приділено *узгодженню назв полів, типів даних і обов'язкових параметрів між Swagger-документацією та реальною реалізацією API, а також пошуку і виправленню типових помилок у документації.*

Це заняття спрямоване на *фінальну підготовку backend-частини проєкту Flora до публікації — від коректно описаних схем і*

ендпоінтів до перевірки роботи документації у Swagger UI та готовності сервера до подальшої інтеграції з клієнтською частиною.

АКТУАЛЬНІСТЬ ЗАНЯТТЯ

Заняття є завершальним етапом підготовки серверної частини проєкту Flora до реального використання, оскільки поєднує документування API, рев'ю коду та розгортання бекенду на хостингу.

ЦІЛІ ЗАНЯТТЯ

Сформувати практичні навички фінальної підготовки backend-проєкту до публікації, документування API за допомогою Swagger та перевірки готовності сервера до інтеграції з клієнтською частиною.

ОСНОВНІ ЗАВДАННЯ

1. Завершити налаштування Swagger-документації для backend-частини проєкту [макету](#) Flora.
2. Перевірити коректність опису всіх ендпоінтів та протестувати роботу документації.

Підготовка до заняття

1. Створення API-документації Swagger

Як ми вже знаємо, у файлі `swagger.json` (або `openapi.json`) усі структури даних описуються в блоці:

```
components → schemas
```

Саме тут задаються **моделі об'єктів**, які приходять у body запиту та повертаються у відповіді сервера.

Схеми потрібні для того, щоб:

- фронтенд чітко бачив, які поля має об'єкт букета;
- Swagger UI міг показувати структуру запитів і відповідей;
- уникати «вгадування» формату JSON.

Для проєкту Flora можна створити кілька схем:

- схема повного об'єкта (Bouquet);
- схема для створення (BouquetCreate);
- схема для оновлення (BouquetUpdate).

У `swagger.json` додається:

```
"components": {  
  "schemas": {  
    "Bouquet": {  
      "type": "object",  
      "required": ["id", "photoURL", "title",  
"description", "price", "favourite"],  
      "properties": {  
        "id": {  
          "type": "string",  
          "example": "64fd123abc456"  
        },  
        "photoURL": {  
          "type": "string",
```

```

        "example": "/photos/bouquet1.jpg"
      },
      "title": {
        "type": "string",
        "example": "Spring bouquet"
      },
      "description": {
        "type": "string",
        "example": "Bouquet with tulips and roses"
      },
      "price": {
        "type": "number",
        "example": 1200
      },
      "favourite": {
        "type": "boolean",
        "example": false
      }
    }
  }
}
}
}
}

```

Це схема **того об'єкта, який повертає API**.

Для **POST** `/api/bouquets` доцільно робити окрему схему:

```

  "BouquetCreate": {
    "type": "object",
    "required": ["title", "description", "price"],
    "properties": {
      "title": {
        "type": "string"
      },
      "description": {

```

```
        "type": "string"
      },
      "price": {
        "type": "number"
      }
    }
  }
}
```

Тут немає `id`, `photoURL`, службових полів. Бо вони створюються сервером.

Для **PATCH / PUT**:

```
"BouquetUpdate": {
  "type": "object",
  "properties": {
    "title": {
      "type": "string"
    },
    "description": {
      "type": "string"
    },
    "price": {
      "type": "number"
    },
    "favourite": {
      "type": "boolean"
    }
  }
}
```

Тут навмисно немає поля `required`. Тому що оновлення — часткове. У розділі `paths` схема використовується через `$ref`. Наприклад для POST:

```
"requestBody": {  
  "required": true,  
  "content": {  
    "application/json": {  
      "schema": {  
        "$ref": "#/components/schemas/BouquetCreate"  
      }  
    }  
  }  
}
```

А для відповіді:

```
"responses": {  
  "201": {  
    "description": "Created",  
    "content": {  
      "application/json": {  
        "schema": {  
          "$ref": "#/components/schemas/Bouquet"  
        }  
      }  
    }  
  }  
}
```

2. Як правильно узгоджувати Swagger-схеми з бекендом

У нашому проєкті (з Joi та Sequelize) дуже важливо дотриматись наступної умови: Swagger-schema має відповідати Joi-схемі валідації й Sequelize-моделі. Наприклад:

якщо в Joi **price** — number → у Swagger теж **number**;
якщо в базі поле називається **photoURL** → у Swagger не можна писати **avatarURL**.

👉 Поля, які сервер генерує сам (id, photoURL, createdAt тощо):

не включаються в create-schema
але обов'язково присутні у response-schema

Типові помилки студентів:

- одна схема на всі запити;
- у Swagger вказані поля, яких немає в API;
- у required вказані поля, які реально необов'язкові при PATCH;
- інші назви полів, ніж у контролері.

Налаштування схем у Swagger для вашого проєкту передбачає:

1. Створити schemas у **components**.
2. Окремо описати об'єкти відповіді, створення, оновлення.
3. Підключити ці схеми в **paths** через **\$ref**.
4. Синхронізувати їх з Joi та моделями.

Фінальна здача проєкту

Фінальна задача проєкту

🔔 Ось ми й дісталися фіналу роботи над вашим проєктом — настав час показати повну його реалізацію для перевірки викладачем та отримати максимально 40 балів за цей етап. Разом максимальну кількість балів за всі три етапи здачі проєкту ви могли отримати 100 балів.

На даному етапі ваш проєкт має покривати функціонал та критерії прийняття всіх пройдених тем та загальні вимоги до проєкту. Деталі – нижче.

Мета етапу

Реалізувати серверну частину Flora: Express-застосунок з коректною архітектурою, підключений до хмарної PostgreSQL через Sequelize, з повним REST API для колекції bouquets (CRUD + Joi-валідація), можливістю завантаження зображень через Multer, задокументований Swagger, задеплоєний на хостинг. На виході — full-stack проєкт з працюючим клієнт-серверним зв'язком, готовий до портфоліо.

Що здаєте

Посилання на backend-репозиторій (з деплоєм)

Посилання на frontend-репозиторій (оновлений на роботу з вашим API)

Посилання на живий backend (хостинг, наприклад Render)

Посилання на живий frontend (GitHub Pages)

Посилання на Swagger UI (/api-docs або аналог)

Підтвердження проходження **код-рев'ю ментором** (мержений PR або коментар)

Що має бути реалізовано в проєкті на даний момент

(6 тематичних блоків)

Блок 1. Архітектура та розгортання backend (Заняття 21–22)

Структура проєкту логічно розділена на шари: запуск застосунку, маршрути, контролери, сервіси, middleware, схеми валідації, helpers/configs або їх аналоги. Точка запуску `server.js` / `index.js`. Підключені middleware для JSON, CORS, централізований обробник помилок. Файл `.env` з портом та іншими параметрами. Сервер запускається без помилок на актуальній LTS-версії Node.

Блок 2. База даних PostgreSQL + Sequelize (Заняття 23–24)

Створений акаунт на Render (або аналог), розгорнута хмарна база. Підключена через Sequelize з обробкою помилки (`process.exit(1)` при фейлі + повідомлення в console). При успіху — "Database connection successful". Модель Bouquet має поля, узгоджені з REST API, Joi-схемами та Swagger: `photoURL` або `photo` — обрати одну назву й використовувати її всюди, `title`, `description`, `price`, `favorite` / `favourite` — також обрати один варіант написання. Усі CRUD-операції перенесені з файлового зберігання на Sequelize-методи.

Блок 3. REST API з валідацією (Заняття 25–26)

Повний CRUD для колекції bouquets — розділений на шари `controllers/services/schemas`:

Метод	Endpoint	Логіка
GET	<code>/api/bouquets</code>	Список усіх, 200
GET	<code>/api/bouquets/:id</code>	Один за id, 200 або <code>{message: "Not found"}</code> 404
POST	<code>/api/bouquets</code>	Joi-валідація обов'язкових полів, 201 або 400
PUT	<code>/api/bouquets/:id</code>	Оновлення переданих полів відповідно до логіки,

		розглянутої на занятті; якщо <code>body</code> порожнє — 400.
DELETE	<code>/api/bouquets/:id</code>	Видалення, 200 або 404
PATCH	<code>/api/bouquets/:id/favorite</code>	Оновлення статусу favorite

Валідація через Joi-схеми в окремих файлах. Централізована обробка помилок (`HttpError.js`). Якщо `validateBody.js` / `HttpError.js` helper'и не використовуються — видалені з проєкту.

Блок 4. Робота з файлами та зображеннями (Заняття 27–28)

Підключений Multer. Створені папки `/temp` (для тимчасових файлів) і `/public/photos` (для постійних). Express налаштований на роздачу статички з `/public`. При створенні букета генерується URL через `gravatar` і зберігається в `photoURL`. PATCH-ендпоінт для оновлення фото: приймає `multipart/form-data`, перекладає файл з `/temp` у `/public/photos` з унікальним ім'ям, повертає 200 з новим URL. Обробка помилок реалізована відповідно до логіки проєкту: 400 при відсутньому або некоректному файлі; 401 — лише якщо в проєкті реалізована авторизація для цього endpoint. Усі назви, пов'язані із зображенням, мають бути уніфіковані: `photoURL` у Sequelize-моделі, Joi-схемах, Swagger і відповіді API. Не використовувати `avatarURL`, якщо endpoint працює із фото букета.

Блок 5. Документація Swagger (`swagger.json` (або `openapi.json`) з розділами `paths` та `components` → `schemas`. (Заняття 29–30)

Окремі схеми для повного об'єкта (`Bouquet`), створення (`BouquetCreate` — без серверних полів), оновлення (`BouquetUpdate` — без `required`). Використання через `$ref`.

Критична вимога: Swagger-схеми синхронізовані з Joi-валідацією та Sequelize-моделями (однакові назви полів,

однакові типи, однакові обов'язкові поля в create-схемі). Swagger UI доступний за URL і коректно відображає всі ендпоінти.

Swagger має описувати саме ті endpoint-и, які реально реалізовані в backend. Не залишати приклади з іншого проєкту або старі назви ресурсів.

Блок 6. Фінальна інтеграція та деплой

Frontend перемкнений з `json-server` на реальний backend. Backend задеплоєний на хостингу (Render або аналог), доступний за публічним URL, стабільно працює. Фінальний PR створений і поданий на код-рев'ю; якщо рев'ю вже проведено — фідбек врахований, PR змержений у `main`. Фінальна версія frontend задеploєна на GitHub Pages і працює зі справжнім API. `README.md` з інструкцією запуску локально (опціонально, але вітається).

Критерії прийняття (обов'язковий мінімум)

Backend запускається локально без помилок.

Підключення до PostgreSQL стабільне.

Усі REST-ендпоінти відповідають (перевіряється через Postman/Swagger UI).

Joi-валідація працює (некоректне body → 400 зі змістовним message).

Swagger UI доступний, і схеми узгоджені з реальним API.

Backend задеploєний на хостингу і доступний за публічним URL.

Frontend працює з реальним backend.

Проведено код-рев'ю ментором або викладачем.

Шкала оцінювання: 40 балів

Блок	Бали	Що оцінюється
Архітектура та розгортання	5	Логічна структура папок, middleware, <code>.env</code> , коректний запуск

PostgreSQL + Sequelize	6	Підключення з обробкою помилок, модель, перенесення CRUD на ORM
REST API з валідацією	10	Повний набір ендпоінтів, коректні коди відповідей, Joi-схеми, розділення на controllers/services/schemas
Робота з файлами	5	Multer, temp → public/photos, роздача статички, унікальні імена файлів
Swagger-документація	5	Окремі схеми для create/update/response, синхронізація з Joi та моделями, працюючий Swagger UI
Фінальний деплой та інтеграція	6	Frontend+backend на хостингу, працюють у зв'язці, код-рев'ю пройдено
Якість коду та послідовність	2	Відсутність <code>console.log</code> , видалені невикористовувані helper'и, код приведений до продакшн-вигляду
Своєчасність здачі	1	У встановлений термін

Типові помилки студентів

Одна Swagger-схема на всі запити — `id` у create, `required` у update.

Назви полів у Swagger не співпадають з контролером (наприклад, `avatarURL` у доці vs `photoURL` у коді).

Joi-валідація в контролері замість окремого middleware — код дублюється між POST і PUT.

Не обробляють помилку підключення до БД → сервер «падає мовчки».

Зберігають завантажені файли одразу в `/public` без проміжного `/temp` — ризик залишити «сміття» при фейлі обробки.

Backend задеплоєний, але frontend все ще ходить у `json-server` → демо «на демо».

`process.exit(1)` забутий при фейлі підключення — сервер запускається, але падає при першому запиті.

Не видаляють `HttpError.js` / `validateBody.js` якщо їх не використовують.